Normalized smoothness coordinates for penalized least-squares trend estimation: a computational study

Heriberto Espino-Montelongo[1*] and Daniela Cortés-Toto[1]

[1*]Universidad de las Américas Puebla, San Andrés Cholula, Puebla, Mexico.

Abstract

Penalized least-squares smoothing is commonly tuned by a nonnegative penalty parameter whose numerical scale varies with the order of differencing. We investigate an alternative computational coordinate: normalized trace smoothness $\boldsymbol{s = S(\lambda) \in [0, 1]}$. For a specified difference order and fixed reference-difference mean, $\boldsymbol{S}$ is strictly increasing, so optimizing an existing selection criterion over $\boldsymbol{s}$ is mathematically equivalent to optimizing it over $\boldsymbol{\lambda}$, with the endpoints represented explicitly. We present the spectral inverse, the derivative transformation for classical cross-validation, and an implementation supporting both search spaces without changing the statistical criteria. Deterministic regression tests compare scores on corresponding penalty and smoothness grids, including nonzero reference means and boundary cases. We also examine the computation supporting repeated candidate evaluations: numerical agreement and conditioning across difference orders, and measured costs of naive dense, cached dense, reusable singular-value, general banded, dense Cholesky, and banded Cholesky solvers under matched workloads. Archived GCV searches and a 48-row Windows BIC check document measured solver costs, selected-score agreement and the limitations of comparing runs under different processor contention. A nonlinear synthetic example illustrates classical selection rules, while an observed carbon-dioxide series demonstrates smoothing across orders at prescribed normalized smoothness. The bounded coordinate improves interpretability, but its inversion adds numerical work; we do not infer faster optimization or improved forecast accuracy without corresponding comparisons. Joint estimation of the reference mean and smoothing parameter is left for future work.

Keywords: normalized smoothness index, penalized least squares, smoothing parameter selection, spectral computation, numerical benchmarking

1 Introduction

Finite-difference penalized least squares (PLS) is an established method for extracting trends from noisy time series Whittaker (1923); Guerrero (2007). Its smoothing parameter $\lambda \geq 0$ is conventionally expressed in penalty units. However, equal numerical penalties need not produce comparable degrees of smoothing for different difference orders. Guerrero's trace-based smoothness measure provides an interpretable description of the resulting smoother Guerrero (2008). Normalizing its range by the number of penalized directions yields a coordinate $s = S(\lambda) \in [0, 1]$, with $s = 0$ for no smoothing and $s = 1$ for the polynomial limiting fit.

The central question of this computational study is whether normalized smoothness can serve as a practical, interpretable coordinate for searching established PLS selection criteria, without changing their statistical meaning or concealing computational costs. We retain both λ- and s-based parameter searches and expose the explicit inverse relationship, analytical derivative transformation, exact endpoints, and finite-precision limitations. This is a reparameterization of known estimators, not a new smoothing criterion or a claim that numerical global optimization becomes automatic.

Efficient implementation matters because a bounded parameter does not make evaluation free. The inverse S^{-1} must be computed at candidate s values, and objective evaluation may require either a complete trend fit or only a spectral trace and residual sum of squares. The banded structure of PLS normal equations can favor an individual fit, whereas a reusable decomposition can favor repeated evaluations Weinert (2007, 2012); Biessy (2026). Accordingly, we supplement the parameterization with numerical agreement, higher-order conditioning, and measured comparisons of fresh and cached dense solves, spectral reuse, and a banded solver under matched workloads. These measurements describe solver costs, *not* a head-to-head timing comparison of end-to-end optimizers in λ and s.

We report mapped-grid equivalence checks and numerical validation of the CV derivative, a single illustrative nonlinear simulation for CV, GCV, AICc, and BIC, and a historical atmospheric carbon-dioxide example at prescribed normalized smoothness. Neither one synthetic realization nor mapped-grid agreement establishes a general statistical ranking or speed advantage. Arbitrary difference orders and fixed nonzero reference means belong to the published PLS family Guerrero (2007); Cortés-Toto et al. (2017); their use is not a novel penalty formulation.

The computational lineage is likewise established. Cortés-Toto et al. Cortés-Toto et al. (2017) cite Weinert's work on Cholesky-based cubic-spline algorithms Weinert (2012), but describe their own CV/GCV/AICc/BIC simulation implementation in R with criteria evaluated explicitly for discrete PLS (Section 3.2). Their study does not establish that those simulations used a Cholesky factorization or compare Cholesky against SVD. Weinert's earlier structured Whittaker–Henderson algorithms Weinert (2007) and his book comparing Cholesky, QR and FFT approaches Weinert (2012) are prior numerical work, not algorithms introduced here.

The numerical experiments condition on a specified mean μ of the penalized differences. The software also provides the plug-in $\mu = \overline{D_d y}$ and conditional Gaussian

marginal-likelihood selection. We display a separate mathematical joint-likelihood profile as future work, but neither joint (λ, μ) estimation, data-adaptive effective degrees of freedom, forecast-horizon validation, nor predictive gains are evaluated in this article.

2 Established model and reusable spectral computation

2.1 Finite-difference penalized least squares

Let $y = (y_1, \ldots, y_n)^\top \in \mathbb{R}^n$ denote an equally spaced series. For an integer $1 \leq d < n$, let $D_d \in \mathbb{R}^{(n-d)\times n}$ represent the dth forward-difference operator; for example, row entries of D_2 are $(1, -2, 1)$. The central estimator is the published penalized least-squares problem

$$\widehat{\tau}_{\lambda,\mu} = \arg\min_{\tau\in\mathbb{R}^n} \left\{ \|y - \tau\|_2^2 + \lambda \|D_d\tau - \mu\mathbf{1}\|_2^2 \right\}, \qquad \lambda \geq 0. \tag{1}$$

where $\mu \in \mathbb{R}$ is a fixed reference mean for the differences. The choice $\mu = 0$ gives the familiar pure penalty. The implementation additionally permits $d = 0$, with $D_0 = I_n$, as a well-defined edge case; the geometric statements below concern $1 \leq d < n$.

Write $Q_d = D_d^\top D_d$ and choose $p \in \mathbb{R}^n$ satisfying $D_d p = \mu\mathbf{1}$. Using zero-based indices $t = 0, \ldots, n-1$, one convenient choice is $p_t = \mu\binom{t}{d}$ because $D_d\binom{t}{d} = 1$. Setting $x = \tau - p$ and differentiating the strictly convex objective gives

$$\widehat{\tau}_{\lambda,\mu} = p + H_\lambda(y - p), \qquad H_\lambda = (I_n + \lambda Q_d)^{-1}. \tag{2}$$

Indeed, $(I_n + \lambda Q_d)x = y - p$ and $I_n + \lambda Q_d$ is positive definite for every finite $\lambda \geq 0$. This affine expression is equivalent to the normal equations $(I_n + \lambda Q_d)\widehat{\tau} = y + \lambda\mu D_d^\top \mathbf{1}$.

2.2 Proof of the d-dimensional kernel

For $1 \leq d < n$, the dth forward difference has the explicit binomial stencil

$$(D_d\tau)_t = \sum_{k=0}^{d} (-1)^{d-k} \binom{d}{k} \tau_{t+k}, \quad t = 0, \ldots, n-d-1. \tag{3}$$

The first $n-d$ columns of D_d form an upper triangular $(n-d)\times(n-d)$ submatrix with nonzero diagonal $(-1)^d$. Hence $\operatorname{rank}(D_d) = n-d$. As $Q_d = D_d^\top D_d$, $x^\top Q_d x = \|D_d x\|^2$, and therefore

$$\ker Q_d = \ker D_d, \qquad \dim\ker Q_d = n - \operatorname{rank}(D_d) = d. \tag{4}$$

The vectors $1, t, \ldots, t^{d-1}$ have vanishing dth forward differences and are linearly independent on $n > d$ nodes. Thus they give a basis of the kernel, proving that exactly d eigenvalues of Q_d vanish in exact arithmetic. Under $\mu = 0$, the limit $\lambda \to \infty$ is the

least-squares projection onto polynomials of degree $\leq d-1$. For instance $d=1$ preserves a constant, $d=2$ a line, and $d=3$ a quadratic. When $\mu \neq 0$, the limiting set is instead the affine space $\{\tau : D_d\tau = \mu\mathbf{1}\}$, which has the same d homogeneous directions but includes a degree-d particular solution. In the implementation $p_t = \mu\binom{t}{d}$.

2.3 Interpretation of higher orders and the spline connection

For a sufficiently smooth function f observed on an equally spaced mesh of width h, Taylor expansion yields $\Delta_h^d f(x_t) = h^d f^{(d)}(x_t) + O(h^{d+1})$. Consequently, under appropriate mesh-refinement assumptions,

$$h^{1-2d}\sum_t (\Delta_h^d f(x_t))^2 \longrightarrow \int |f^{(d)}(x)|^2\,dx. \tag{5}$$

This is a mathematical bridge to roughness penalties in smoothing splines. A Whittaker-type trend smoother penalizes differences of fitted ordinates directly, while a P-spline instead penalizes differences of coefficients in a chosen spline basis; these are related ideas but not identical estimators. For a continuous penalty weight λ_{cont}, the corresponding discrete coefficient is scaled approximately as $\lambda_{\text{disc}} = \lambda_{\text{cont}} h^{1-2d}$.

The order d is an integer representing the order of the derivative being approximated; it is not a continuous interpolation between derivative orders. Comparing different d values is a change of regularization model, so equal numerical λ does not imply equal smoothing. It is useful to examine orders $d = 1, \ldots, 5$ under a common normalized trace-based index; however, arbitrary orders and polynomial nullspaces have long been known Weinert (2007); Biessy (2026). Larger d can be ill-conditioned because the binomial stencil has growing coefficients and the positive eigenvalues may span many orders of magnitude. Neither higher d nor higher polynomial degree guarantees a better trend or numerical stability.

2.4 Null space, SVD, and limiting trends

The rows of D_d are independent in exact arithmetic, so $\operatorname{rank}(D_d) = n - d$ and $\dim\ker D_d = d$. The null space consists of the sampled polynomials of degree at most $d-1$. Let $D_d = U\Sigma V^\top$ be a full SVD. With nonnegative eigenvalues δ_j of Q_d, ordered so that exactly d are zero, the smoother becomes

$$H_\lambda = V\,\operatorname{diag}\left(\frac{1}{1+\lambda\delta_1}, \ldots, \frac{1}{1+\lambda\delta_n}\right) V^\top. \tag{6}$$

Therefore the transformed observations $c = V^\top(y-p)$ can be computed once, and every new penalty value changes only the scalar attenuation factors $(1+\lambda\delta_j)^{-1}$. At $\lambda = \infty$ the exact limiting fit is $p + P_{\ker D_d}(y-p)$, rather than a computation using an arbitrarily large finite penalty.

For fixed μ, the effective degrees of freedom are $\text{EDF}(\lambda) = \operatorname{tr}(H_\lambda) = \sum_j (1+\lambda\delta_j)^{-1}$. We report Guerrero's trace-based smoothness $S_G = 1 - \operatorname{tr}(H_\lambda)/n$ alongside

the normalized coordinate

$$S(\lambda) = \frac{n - \operatorname{tr}(H_\lambda)}{n - d} \in [0, 1], \quad S(0) = 0, \quad S(\infty) = 1. \tag{7}$$

For $1 \le d < n$, this coordinate is strictly increasing for finite λ because $S'(\lambda) = (n-d)^{-1} \sum_{\delta_j > 0} \delta_j / (1 + \lambda \delta_j)^2 > 0$. The normalization changes the scale of the original index but not the ordering of smoothness levels Guerrero (2008).

2.5 Interpretation of fixed nonzero differences

For finite λ, specifying $\mu \neq 0$ shifts the estimator through the reference vector $p_t = \mu\binom{t}{d}$ in Eq. (2); it does not change the smoothing matrix H_λ. As $\lambda \to \infty$, the fitted differences satisfy $D_d \widehat{\tau} = \mu \mathbf{1}$, so the limit is a polynomial of degree at most d rather than at most $d - 1$ when $\mu \neq 0$. For example, a prescribed nonzero first-difference mean admits a linear limiting trend, while a prescribed nonzero second-difference mean admits a quadratic one. These are known consequences of the difference penalty and are used here to check implementation correctness, not to claim a newly introduced family of trends. All degrees-of-freedom formulas below condition on this fixed numerical μ.

2.6 Analytical likelihood profiling: a deferred extension

For completeness, we retain the key formulas motivating a separate likelihood-based extension, without incorporating it into the computational experiments below. Assume a Gaussian model with independent observation errors of variance σ^2 and latent centered dth differences of variance σ^2/λ, where $\lambda > 0$. Write $r = n - d$, let $\delta_j > 0$ and v_j denote the r positive-penalty eigenvalues and corresponding eigenvectors of Q_d, and put

$$p_t^{(1)} = \binom{t}{d}, \qquad z_j = v_j^\top y, \qquad b_j = v_j^\top p^{(1)}, \qquad w_j(\lambda) = \frac{\lambda \delta_j}{1 + \lambda \delta_j}.$$

Eliminating the unpenalized polynomial coordinates and profiling out the observation variance gives the marginal negative log-likelihood criterion, up to constants,

$$J(\lambda; \mu) = r \log \left[\frac{1}{r} \sum_{j=1}^{r} w_j(\lambda)(z_j - b_j \mu)^2 \right] - \sum_{j=1}^{r} \log w_j(\lambda). \tag{8}$$

For a *fixed positive* λ, the corresponding optimal difference mean has the explicit weighted formula

$$\widehat{\mu}(\lambda) = \frac{\sum_{j=1}^{r} w_j(\lambda) b_j z_j}{\sum_{j=1}^{r} w_j(\lambda) b_j^2}. \tag{9}$$

This reduces the prospective two-parameter optimization to a one-dimensional profile problem:

$$\widehat{\lambda} = \arg\min_{\lambda > 0} J\big(\lambda; \widehat{\mu}(\lambda)\big). \tag{10}$$

These identities are a mathematical outline, *not* a tested joint-MLE algorithm or a claim of new likelihood theory Guerrero (2007); Biessy (2026). In particular, the observed-difference plug-in $\mu = \overline{D_d y}$ is not the joint likelihood estimate at general positive λ. The public implementation currently supports likelihood selection conditional on a fixed or plug-in μ, not this joint profile. Identifiability, numerical minimization, inference, and comparative performance for the joint estimator are left to future work. The remaining sections evaluate only fixed-μ smoothing and its computational costs.

3 Classical parameter selection and numerical implementation

3.1 Selection criteria

For a fixed numerical value of μ, define $e_\lambda = y - \widehat{\tau}_{\lambda,\mu}$, $\mathrm{RSS}_\lambda = \|e_\lambda\|_2^2$, $h_{ii} = (H_\lambda)_{ii}$, and $\nu_\lambda = \mathrm{tr}(H_\lambda)$. The classical selectors examined in the single synthetic illustration, following Cortés-Toto et al. (2017), are represented, up to constants irrelevant to an optimizer, by

$$\mathrm{CV}(\lambda) = \frac{1}{n}\sum_{i=1}^{n}\left(\frac{e_{\lambda,i}}{1-h_{ii}}\right)^2, \tag{11}$$

$$\mathrm{GCV}(\lambda) = \frac{\mathrm{RSS}_\lambda/n}{(1-\nu_\lambda/n)^2}, \tag{12}$$

$$\mathrm{AICc}(\lambda) = \log(\mathrm{RSS}_\lambda/n) + \frac{2\nu_\lambda + 1}{n - \nu_\lambda - 2}, \tag{13}$$

$$\mathrm{BIC}(\lambda) = \log(\mathrm{RSS}_\lambda/n) + \frac{\nu_\lambda \log n}{n}. \tag{14}$$

The grid-based selection procedure checks denominators, excludes undefined candidates, and compares the exact infinite-penalty candidate. The implementation also supports derivative-based refinement for classical CV and the bounded smoothness search introduced below. In particular, a near-interpolating BIC minimum can be driven by the log(RSS) boundary, as our example illustrates. We do not interpret a finite-grid BIC minimum close to zero as a universally reliable statistical choice.

Guerrero Guerrero (2007) also considers the feasible plug-in $\widehat{\mu} = \mathrm{mean}(D_d y)$. This option is available *for fitting*; however, the formula for leave-one-out CV in (11) presumes that μ remains fixed under deletion. The current public selectors intentionally require a fixed numerical μ. Likewise, $\mathrm{tr}(H_\lambda)$ is a conditional EDF and is not automatically the full degrees of freedom of a procedure that estimates μ from the same observations.

3.2 Smoothing correction, inversion, and exact endpoints

With the full orthogonal eigenbasis of the penalty, $Q_d = V\,\mathrm{diag}(\delta_1, \ldots, \delta_n)V^\top$, the $n-d$ positive eigenvalues are $\delta_j = \sigma_j(D_d)^2$, and the remaining d are zero. Consequently,

the spectral eigenvalues of $H_\lambda = (I + \lambda Q_d)^{-1}$ are

$$\alpha_j(\lambda) = \begin{cases} (1 + \lambda\delta_j)^{-1}, & 1 \le j \le n - d, \\ 1, & n - d < j \le n. \end{cases} \tag{15}$$

This exposes both the exact limiting polynomial directions and the scalar attenuation of the penalized directions. If $\nu(\lambda) = \operatorname{tr}(H_\lambda) = d + \sum_{j=1}^{n-d} \alpha_j(\lambda)$, the original trace-based Guerrero index

$$S_G(\lambda) = 1 - \frac{\nu(\lambda)}{n} \quad \text{ranges from 0 to } 1 - \frac{d}{n}. \tag{16}$$

The upper bound is $1 - d/n$, *not* $1 - n/d$. The library also uses a normalized correction:

$$S(\lambda) = \frac{S_G(\lambda)}{1 - d/n} = \frac{n - \nu(\lambda)}{n - d} \in [0, 1]. \tag{17}$$

The map is strictly increasing for $\lambda \in [0, \infty)$ because

$$S'(\lambda) = \frac{1}{n - d} \sum_{j=1}^{n-d} \frac{\delta_j}{(1 + \lambda\delta_j)^2} > 0. \tag{18}$$

The exact endpoints are $S(0) = 0$ and $S(\infty) = 1$; for any interior target, a one-dimensional bracketed solver inverts this map. Equal normalized smoothness makes trace scales comparable across d; it does not make the underlying trend geometries or roughness penalties identical.

3.3 Where the computational acceleration comes from

For fixed μ put $p_t = \mu\binom{t}{d}$ and $c = V^\top(y - p)$. One common projection computes c; after that, for each candidate λ,

$$\operatorname{tr}(H_\lambda) = d + \sum_{j=1}^{n-d} \alpha_j(\lambda), \tag{19}$$

$$\|y - \widehat{\tau}_{\lambda,\mu}\|^2 = \sum_{j=1}^{n-d} (1 - \alpha_j(\lambda))^2 c_j^2. \tag{20}$$

Thus a sweep of M values can obtain trace, normalized smoothness, GCV, AICc, BIC and related scalar profiles in $O(Mn)$ *after* the shared eigendecomposition and $O(n^2)$ projection. A full reconstructed trend remains $O(n^2)$ per penalty, and exact ordinary CV leverage diagnostics also involve the spectral basis. The full dense SVD has $O(n^3)$ preprocessing time and $O(n^2)$ storage. This decomposition/reuse trade-off explains the potential responsiveness of repeated-criteria explorations without claiming unconditional superiority to banded linear solvers Weinert (2007); Biessy (2026).

Large batches of spectral attenuation operations are naturally vectorizable. Dense linear algebra can also benefit from optimized multithreaded BLAS on CPUs. In principle, a GPU backend could parallelize matrix decompositions and batched matrix operations, but host/device transfer, synchronization, and initialization costs may negate a benefit for short series. We found `numpy` and `scipy.linalg.svd` in the package currently reviewed, not a released GPU solver; therefore no GPU speedup, power saving, or measured device execution is reported. A future benchmark should record CPU and GPU models, precision, BLAS and accelerator versions, transfer time, warmups, memory, and the number of candidate penalties.

3.4 Selecting a criterion in normalized smoothness coordinates

The monotonicity in Eq. (18) permits a change of optimization coordinate without changing the underlying PLS estimator or statistical selection criterion. Write $r = n - d$, let $\delta_1, \ldots, \delta_r$ denote the strictly positive penalty eigenvalues, and define

$$s = S(\lambda) = \frac{1}{r} \sum_{j=1}^{r} \frac{\lambda \delta_j}{1 + \lambda \delta_j}, \qquad \Lambda(s) = S^{-1}(s), \quad 0 < s < 1. \tag{21}$$

Since

$$S'(\lambda) = \frac{1}{r} \sum_{j=1}^{r} \frac{\delta_j}{(1 + \lambda \delta_j)^2} > 0 \qquad (\lambda \geq 0), \tag{22}$$

S is continuous and strictly increasing, taking the extended endpoints $S(0) = 0$ and $S(\infty) = 1$. We define $\Lambda(0) = 0$ and $\Lambda(1) = \infty$ exactly. These formulas apply for any admissible order $0 \leq d < n$, provided the positive penalty spectrum is numerically resolved.

For an existing criterion $C(\lambda; \mu)$ conditional on a fixed reference mean, introduce $\widetilde{C}(s; \mu) = C(\Lambda(s); \mu)$. The same ideal optimization problem can be written as

$$\widehat{s} \in \arg\min_{s \in [0,1]} \widetilde{C}(s; \mu), \qquad \widehat{\lambda} = \Lambda(\widehat{s}). \tag{23}$$

Here inadmissible boundary values retain their original definition, including an undefined ordinary CV score at $\lambda = 0$; the correspondence between global minimizers is set-valued in the presence of ties. This is an exact reparameterization, not a new criterion or a proof that a finite numerical search locates the global optimum. Equal s across difference orders means equal *normalized conditional trace smoothness*, not equal trend geometry, variability, or forecast accuracy.

For a differentiable criterion and $0 < s < 1$, the inverse-function theorem and the chain rule give

$$\frac{d\widetilde{C}}{ds} = \frac{C'(\lambda)}{S'(\lambda)} = \frac{dC/d\log\lambda}{dS/d\log\lambda}, \qquad \frac{dS}{d\log\lambda} = \frac{1}{r} \sum_{j=1}^{r} \frac{\lambda \delta_j}{(1 + \lambda \delta_j)^2} > 0, \quad \lambda = \Lambda(s). \tag{24}$$

Thus interior stationary points coincide in the two parameterizations. The public implementation exposes the optional `search_space="smoothness"` in `select_lambda`, while `search_space="lambda"` remains the default. For classical fixed-μ CV, it scans a uniform and a logit-spaced set of s values, refines sampled derivative sign-change brackets and score valleys, and compares evaluated criterion values at those candidates and exact boundaries. For GCV, AICc, BIC, and conditional Gaussian MLE the option evaluates a grid in s, using the same score as in λ. A user-supplied `grid` is interpreted in the selected coordinate. The returned `lambda_` remains a penalty; `smoothness` returns its normalized trace index, `grid` records evaluated penalties, and `search_grid` records the requested coordinates. Only Gaussian MLE supports the previously defined observed-difference plug-in for μ; the new coordinate does not jointly estimate (λ, μ).

The inverse is obtained with a safeguarded root finder in $\log \lambda$, reusing the spectral representation. Every interior smoothness evaluation therefore incurs an additional scalar inversion, typically several $O(r)$ spectral evaluations, before the unchanged criterion score. Moreover $dS/d\log\lambda$ tends to zero at both limits, so derivative conversion can be poorly conditioned near $s = 0$ or $s = 1$. Interior values requiring an unrepresentable floating-point penalty are rejected rather than silently treated as endpoints. A bounded search interval does not, by itself, establish a speed or accuracy advantage over direct optimization in $\log \lambda$.

For completeness, the prospective joint likelihood profile in Eq. (10) could also be expressed in this coordinate as

$$\widehat{s}_{\text{joint}} = \arg\min_{0<s<1} J\big(\Lambda(s); \widehat{\mu}(\Lambda(s))\big), \qquad \widehat{\lambda}_{\text{joint}} = \Lambda(\widehat{s}_{\text{joint}}). \tag{25}$$

Equation (25) describes an *unimplemented research extension*, not a result of the conditional-MLE selector or of the benchmarks below. Boundary candidates and identifiability would require separate treatment in a full joint-estimation study.

3.5 Reusable computation and cost

The reusable `TrendSmoother(n, order=d)` implementation builds the difference operator and its SVD once. Each candidate λ then changes only the spectral attenuation in Eq. (6). The public functions used in this study expose fitted trends, effective degrees of freedom, normalized smoothness, and classical criterion evaluation. The accompanying scripts specify their seeds, inputs, numerical tolerances, and host-recorded timings. The computational experiments, rather than the existence of a Python interface, are the object of evaluation.

The computational trade-off must distinguish decomposition from reuse. A dense full SVD costs $O(n^3)$ preprocessing and $O(n^2)$ memory; after projection, EDF, smoothness, GCV, AICc, and BIC summaries cost $O(n)$ per penalty candidate, while full reconstruction and exact CV leverage scores require $O(n^2)$ each. Smoothness-coordinate selection adds inversion work to each new interior candidate and is therefore not covered by the solver-only timing comparisons. In contrast, the banded structure of $I_n + \lambda Q_d$ allows efficient specialized direct solvers Weinert (2007). Thus the phrase *fast computation* means repeated spectral evaluation under a stated workload,

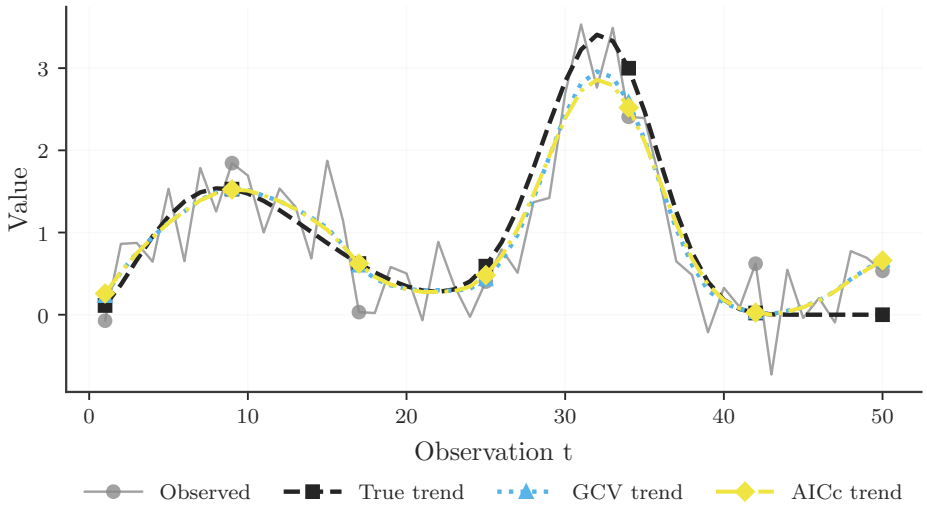


Fig. 1 A synthetic nonlinear case based on Cortés-Toto et al. Cortés-Toto et al. (2017). The curves show *in-sample* estimates and known simulated truth, not out-of-sample forecasts. Source data are regenerated by `figures/make_figures.py`.

not a blanket claim that SVD beats banded factorization. A separate reproducible benchmark in the repository measures setup and fit times independently.

4 Reproducible numerical illustration

4.1 Source-inspired nonlinear design

To demonstrate the interface without introducing an external data source, we reproduce the nonlinear simulation family of Cortés-Toto et al. Cortés-Toto et al. (2017):

$$\tau(t) = 0.6 f_{\mathrm{Beta}(30,17)}(t/n) + 0.4 f_{\mathrm{Beta}(3,11)}(t/n), \quad t = 1, \ldots, n. \tag{26}$$

We take $n = 50$, add independent normal errors with standard deviation 0.5, omit seasonality, and fix the pseudo-random seed at 19. These choices define one illustrative replication, not a full reproduction of the article's factorial simulation. We fit $d = 2$, $\mu = 0$, and select λ from the same 121-point geometric grid augmented with the exact $\lambda = \infty$ candidate under each criterion. The latent trend in (26) is known, allowing an *ex post* trend-recovery RMSE; it is not a future forecast error.

As shown in Table 1, CV and GCV agree in this particular realization, whereas AICc selects a somewhat smoother fit. BIC selects a penalty extremely close to zero on the inspected finite grid; its apparent interpolation tendency illustrates a known practical boundary concern and is not evidence that BIC is generally ineffective. The simulated recoveries are descriptive; no general method ranking follows from a single seed.

Table 1 Selected penalty and normalized smoothness for the case in Fig. 1. Trend RMSE is computed against the known latent simulation only.

Criterion	$\widehat{\lambda}$	S	Trend RMSE
CV	6.17	0.777	0.242
GCV	6.14	0.777	0.242
AICC	9.74	0.805	0.248
BIC	2.45e-07	0.000	0.454

4.2 Algebraic and numerical checks

An independent check compares the SVD implementation with the direct normal equations for $n = 50$, $\lambda = 30$, $d \in \{1, 2, 3\}$ and $\mu \in \{0, 0.15\}$ on a second fixed-seed Gaussian sample. The maximum componentwise absolute difference is 2.01×10^{-12} across these six cases, and the limiting constraint $D_d \widehat{\tau}_{\infty,\mu} = \mu \mathbf{1}$ holds to maximum error 4.42×10^{-12}. Both values result from the supplied deterministic regression script in IEEE double precision; they verify the tested algebra rather than establish stability for all n, d, λ. In addition, the library regression tests evaluate matched candidate penalties and normalized-smoothness coordinates for $d \in \{0, 1, 2, 4\}$ at a fixed nonzero mean, comparing the original criterion scores after inverse mapping. They also check the smoothness-space CV derivative against centered finite differences. These tests establish numerical agreement at the checked inputs, not faster optimization or identical outcomes of two independently sampled finite grids. Higher differencing orders can be ill-conditioned, and systematic machine-dependent speed measurements remain a distinct stage of the study.

5 Measured computational cost and repeated parameter evaluation

5.1 Experimental design and independent baselines

Numerical correctness and elapsed time are distinct properties. We therefore benchmark the same PLS estimator at matched (n, d, μ) and identical candidate penalties, using four implementations: (i) a deliberately naive dense solver that reconstructs D_d, $Q_d = D_d^\top D_d$, and solves the normal equations at every candidate; (ii) a denser but fairer baseline that caches Q_d and repeats its dense solve; (iii) the reusable full-SVD spectral implementation of `TrendSmoother`; and (iv) a banded solver that assembles Q_d directly in its sparse-band storage and uses a structure-aware linear solver. The last comparison is crucial: beating dense normal equations does not imply beating specialized PLS solvers Weinert (2007, 2012).

Banded factorization and benchmark scope.

For every finite $\lambda \geq 0$, the normal-equation matrix $A_\lambda = I_n + \lambda D_d^\top D_d$ is symmetric positive definite, since $x^\top A_\lambda x = \|x\|_2^2 + \lambda \|D_d x\|_2^2 > 0$ for any nonzero x. It has half-bandwidth d, so a banded Cholesky factorization $A_\lambda = L_\lambda L_\lambda^\top$ is a valid structure-aware

Table 2 Leading arithmetic complexity for M reconstructed trends of length n and fixed order d. Setup is counted and no exact operation counts or GPU measurements are asserted.

Method	Setup	Repeated full fits
Fresh dense solves	rebuilt per λ	$O(Mn^3)$
Cached dense Q_d	dense penalty matrix	$O(Mn^3)$
Spectral SVD	$O(n^3)$	$O(Mn^2)$
Direct banded solver	$O(nd^2)$	$O(Mnd^2)$

solution method. Its factor generally must be recomputed when λ changes, whereas a spectral basis can be reused across penalties. This is established computational prior art Weinert (2007, 2012).

Importantly, the *banded* timings in the four-method tables below come from `scipy.linalg.solve_banded`, a general banded linear-system solver in `paper/benchmarks/run_benchmarks.py`; they are *not* measured Cholesky timings. The separate library backend `PreparedPLSSolver(method="cholesky")` in `src/trendestimation/solvers.py` instead calls `scipy.linalg.cholesky_banded` followed by `scipy.linalg.cho_solve_banded` to produce full fitted trends. Neither its timings nor Weinert's QR or FFT methods are included in these four-method tables. We therefore make no measured Cholesky-specific speed claim from this experiment.

For every case a deterministic Gaussian sequence is generated, μ is held at either zero or a fixed nonzero value, and λ spans 10^{-3} to 10^3 on a geometric grid. The `quick` experiment varies $n \in \{40, 100, 180\}$, $M \in \{1, 16, 64\}$ with $d = 2$ and adds $d \in \{1, 3, 5\}$ at $n = 100$, $M = 24$, together with a nonzero-μ case. The `full` experiment reaches $n = 400$, $M = 256$ and additional difference orders. All results include one-time setup (not only post-SVD solves); we record medians, extrema, and median absolute deviations over repeated independent runs. We bound maximum disagreement against an independent banded solve for *every* tested penalty. Runs use a declared random seed, source revision, machine metadata, NumPy/SciPy versions and a stated BLAS-thread limit. The code and machine-readable outputs are confined to `paper/benchmarks/`.

5.2 End-to-end full-fit timing results

The crossover between methods should be evaluated using *end-to-end* times, including the SVD setup cost, for both a single smoothing parameter and a repeated-λ sweep. A ratio greater than one denotes a speed advantage over the naive baseline on the reported machine, not a universal speedup. In particular, the structured banded baseline can outperform a reusable SVD for full trend reconstruction.

Table 3 Host-measured end-to-end median times in milliseconds, including one-time setup, for M full fitted trends. Every method computes the same trends for the same n, $d = 2$ and $\mu = 0$; R is the naive-fresh-dense time divided by the spectral time. Values and the numerical parity check are generated from the raw CSV files, not entered manually.

n	M	Fresh	Cached	Spectral	Banded	R
40	1	0.029	0.027	0.204	0.123	0.14
40	64	1.388	0.739	1.302	1.032	1.07
100	1	0.075	0.073	0.801	0.281	0.09
100	64	4.589	2.642	2.013	1.532	2.28
200	1	0.471	0.350	3.580	0.535	0.13
200	64	20.984	10.348	4.982	2.017	4.21
400	1	3.192	3.217	20.165	1.103	0.16
400	64	196.554	106.894	20.885	3.188	9.41

Table 4 Median end-to-end times (milliseconds) to evaluate M effective-degrees-of-freedom traces, including setup. These entries measure only scalar $\mathrm{tr}(H_\lambda)$ quantities, not reconstructed trends.

n	M	Fresh	Cached	Spectral	Banded
40	8	0.213	0.150	0.203	0.322
100	8	1.059	0.832	0.801	1.072
100	32	4.220	3.307	0.885	3.615
180	32	15.914	12.208	2.830	10.114
250	64	92.614	59.421	6.090	38.391

5.3 Scalar smoothness and EDF are different workloads

To evaluate $\mathrm{tr}(H_\lambda)$ repeatedly, the spectral method needs only $n - d$ positive eigenvalues:

$$\mathrm{tr}(H_\lambda) = d + \sum_{j=1}^{n-d} (1 + \lambda\delta_j)^{-1}.$$

This is $O(Mn)$ work *after* spectral setup, not $O(Mn^2)$ reconstructed trends. An independent second experiment compares repeated dense computations of $\mathrm{tr}[(I + \lambda Q_d)^{-1}]$, a banded solver applied to all n canonical right-hand sides, and the spectral sum. The experiment checks algebraic equivalence separately from timed regions. Even when the spectral trace is the most efficient *marginal* score after setup, the $O(n^3)$ SVD cost can still dominate small workloads.

Measurement provenance: Platform: Windows-10-10.0.26100-SP0; CPU: Intel64 Family 6 Model 183 Stepping 1, GenuineIntel; Logical CPUs: 32; NumPy: 2.4.6; SciPy: 1.17.1; BLAS threads: 1; Repetitions: 7; Preset: full; Revision: local-unrecorded.
Maximum observed absolute trend disagreement: $4.32e - 10$; maximum EDF trace disagreement: $5.48e - 12$.

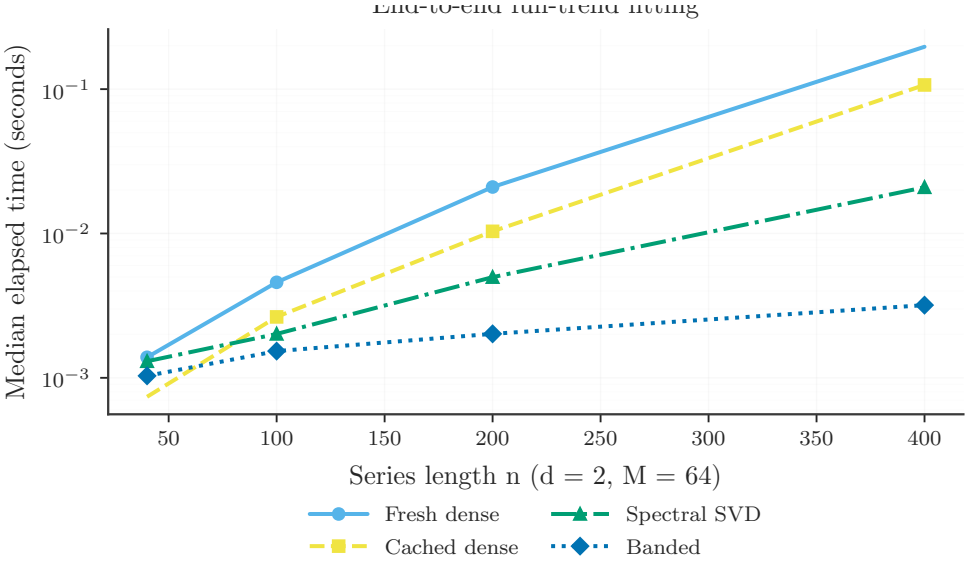


Fig. 2 End-to-end median times for $M = 64$ fitted curves, including setup, generated on the executing host from the raw CSV files. These are hardware-specific measurements, not an assumption that spectral reconstruction is the universally fastest method.

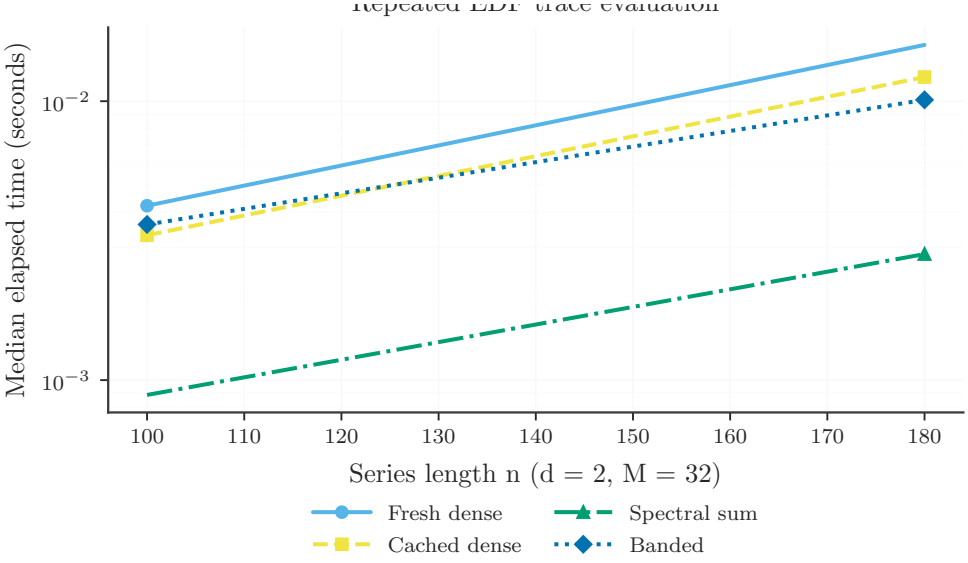


Fig. 3 Repeated effective-degrees-of-freedom trace evaluations ($M = 32$), including preprocessing. The quantity is scalar $\mathrm{tr}(H_\lambda)$, not a full trend reconstruction.

5.4 Interpreting speed and computational cost

A timed speed ratio is defined by the two end-to-end medians, for example $R_{\text{dense/spectral}} = t_{\text{dense}}/t_{\text{spectral}}$. A value above one favors the spectral approach for the tested workload, not for arbitrary data lengths or hardware. CPU time, wall time, arithmetic complexity and energy consumption are different quantities. CPU numerical libraries already use optimized native linear algebra, but this experiment does not establish GPU execution or electricity use. An explicit device and power measurement would be necessary to make either claim. Results are intentionally generated only through a manual command, so the manuscript does not inadvertently contain timings from a different machine presented as those of its authors.

5.5 Six backends for full fits versus actual GCV grid selection

The preceding four-method tables concern fixed-penalty trend fitting and scalar EDF calculations, not a complete parameter search. A separate, reproducible companion experiment compares six numerical approaches: fresh and cached dense solves, reusable full SVD, the general banded solver, dense Cholesky, and *banded Cholesky*. The new Cholesky measurements use `cho_factor`/`cho_solve` or `cholesky_banded`/`cho_solve_banded`, respectively; we do not retroactively relabel the general banded solver as Cholesky. All return numerically agreeing trend fits under the same fixed (n, d, μ, λ), including nonzero μ.

For *actual* classical GCV grid search, each solver evaluates $\mathrm{GCV}(\lambda) = n\|y - \widehat{\tau}_\lambda\|_2^2/[n - \mathrm{tr}(H_\lambda)]^2$ and selects the smallest finite score over its supplied candidate grid. For fixed μ, $A_\lambda \widehat{\tau} = y + \lambda\mu D_d^\top \mathbf{1}$, $A_\lambda = I + \lambda D_d^\top D_d$, and $H_\lambda = A_\lambda^{-1}$. Consequently, changing the fixed reference mean alters fitted values and criterion values but not H_λ, its trace, or the coefficient matrix. For Cholesky and dense/banded direct methods, the trace calculation requires additional operations beyond a full trend solve. In contrast, SVD reuses the spectral eigenvalues for EDF and the already projected observations for the grid scores.

The `quick` reference experiment considers $d \in \{1, 2, 3, 4\}$, $M \in \{5, 11\}$, $R \in \{1, 4\}$, and fixed $\mu \in \{0, 0.15\}$ at $n = 48$. It compares equally spaced log-penalty points with uniformly spaced normalized-S points mapped to the *same finite penalty endpoints*. The common $S \to \lambda$ inversion cost is timed separately and deliberately excluded from the method-specific numbers above. An S grid and a log-penalty grid need not contain the same interior candidates or select the same discrete optimum; this does not contradict continuous reparameterization invariance. Exact $S = 0$ and $S = 1$ are supported elsewhere in the library but excluded from the finite-system comparison to avoid limiting/singular computational branches.

In the recorded scenarios, reusable SVD was faster for the *end-to-end GCV selection* cases in Table 5, while separate full-fit-only measurements favor banded Cholesky for many repeated reconstructions. Reusing the *same* grid across multiple independent series can amortize the preprocessing (Table 6); Cholesky factors do *not* carry over across different penalty values. These are descriptive measurements from a GitHub-hosted Linux runner with Python 3.11, NumPy 2.4.6, SciPy 1.17.1, one BLAS thread,

Table 5 Measured end-to-end classical GCV grid-search time (milliseconds) on the recorded GitHub Actions Linux runner, including setup and 11 log-penalty candidate scores, with one independent series ($n = 48$, $R = 1$), warm preparation, one BLAS thread, and three timing blocks. The reference mean μ is fixed in each comparison. BChol is banded Cholesky; the separate general-banded route and dense Cholesky are available in the underlying six-method CSV. Lower is faster in this sampled workload.

d	μ	SVD (ms)	BChol (ms)	SVD/BChol
1	0.00	0.630	1.205	0.52
1	0.15	0.621	1.204	0.52
2	0.00	0.622	1.287	0.48
2	0.15	0.626	1.259	0.50
3	0.00	0.628	1.336	0.47
3	0.15	0.623	1.363	0.46
4	0.00	0.630	1.421	0.44
4	0.15	0.621	1.413	0.44

Table 6 Reusing candidate-specific preprocessing across four independently simulated series ($n = 48$, $d = 2$, $M = 11$, $\mu = 0$), end-to-end GCV search time including preparation on the same Linux runner. Cold rebuilds for each series; warm prepares once and reuses information at the same penalty candidates. Speedup is cold/warm.

Method	Cold (ms)	Warm (ms)	Cold/Warm
SVD	2.451	0.888	2.76
Banded Cholesky	5.167	1.830	2.82
General banded	6.275	3.234	1.94
Dense Cholesky	3.436	1.507	2.28

three repeats, and one seeded synthetic data design, not statistical evidence of a universally fastest backend. Raw records and environment provenance are archived in `paper/benchmarks/grid_reuse_reference/`. A separate six-way full-fit table with its own provenance is available under `paper/benchmarks/cholesky_reference/`. The author's subsequent Windows-VM `full` run is not included here because its CSV files have not yet been archived in the repository.

The library therefore retains *spectral SVD* for `TrendSmoother` and for CV/GCV/AICc/BIC/conditional MLE selection, while its *trend-only* convenience solver defaults to banded Cholesky. This is a workload-specific engineering choice, not a change to the objective, selection criterion, or theory.

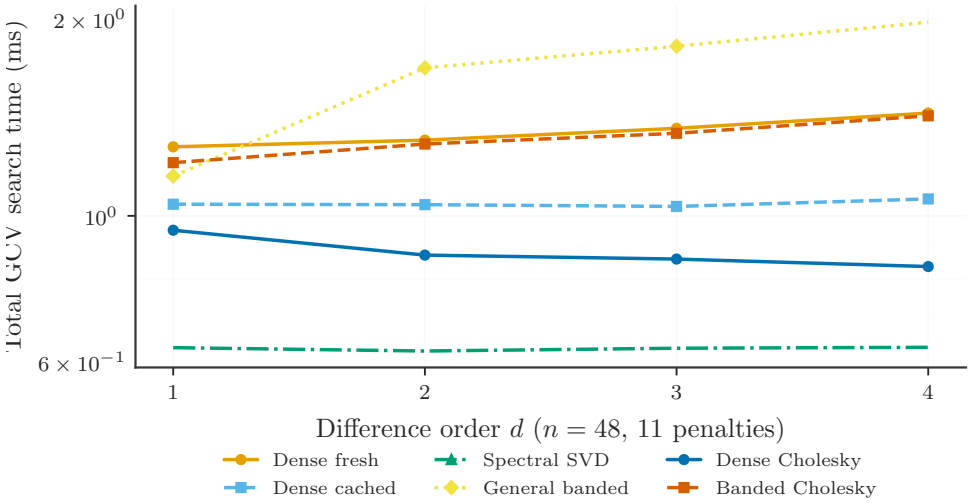


Fig. 4 Measured end-to-end classical GCV grid selection for six numerical solvers and finite-difference order $d = 1, \ldots, 4$, $n = 48$, $M = 11$ positive finite log-penalty candidates, fixed $\mu = 0$, one series, and warm setup, on the recorded GitHub-hosted Linux machine. Each method computes its own EDF; general banded and banded Cholesky are distinct factorizations. The vertical axis is logarithmic.

5.6 AwesomeFigures visualization of computational cost and five selection criteria

Figures 4–8 are generated from archived machine-recorded CSV files through the `awesomefigures` Python library, with vector PDF export at the measured text width and typography of the official Springer Nature manuscript. Browser-compatible SVG variants accompany the documentation. No benchmark values are generated during plotting.

The comparison of statistical selection criteria also includes Gaussian profiled marginal maximum likelihood (MLE) already available as `criterion="mle"`, alongside fixed-mean CV, GCV, AICc and BIC. This MLE profiles the observation variance, integrates the penalized latent coordinates and excludes the unpenalized polynomial null space. It conditions on a fixed numerical difference-reference mean μ, not on an estimated joint (μ, λ) likelihood. MLE is a *criterion*, not a seventh matrix factorization. At $\lambda = 0$ the Gaussian latent model is improper and MLE assigns an infinite score. Raw scores across the five criteria are not comparable.

The search-coordinate experiment uses the same 16 candidate smoothness targets $(0, 0.1, \ldots, 1$ plus $0.95, 0.99, 0.999, 0.9999, 0.999999)$. Mapped penalty first computes $S^{-1}(s_j)$ and reuses the resulting penalties for all five criteria. Native S supplies those same targets to each public selector, repeating the inversions; both score identical candidates up to numerical tolerance. An independent log-penalty grid has 16 values

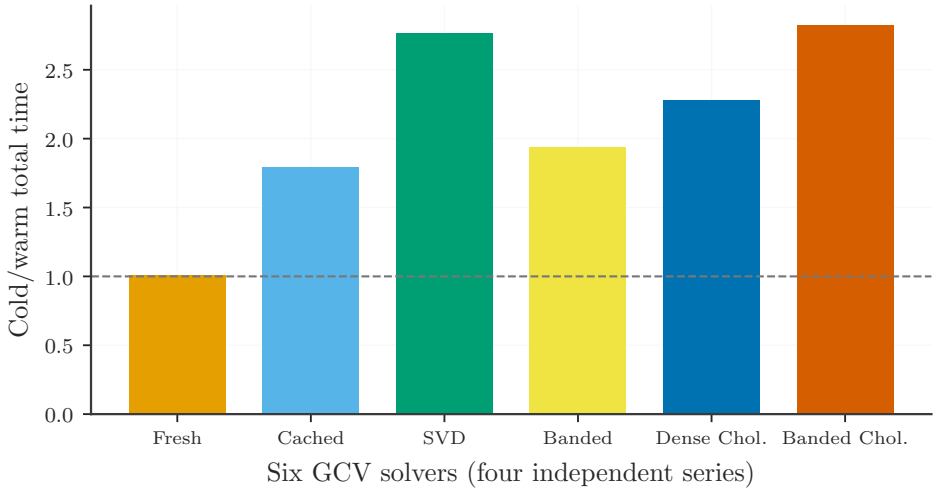


Fig. 5 Ratio of measured cold to warm complete GCV selection time for four independent series revisiting identical finite penalties ($n = 48$, $d = 2$, $M = 11$, fixed $\mu = 0$). Ratios above one favor shared preprocessing; a Cholesky factor may be reused for a repeated penalty but not for a different one.

including both endpoints but different interior candidates, so finite-grid minima need not coincide. The mapped and native-S selection times exclude shared SVD setup, and the one-time shared mapping cost is recorded separately. The scope differs from the six-backend GCV experiment above.

Both studies also evaluate fixed $\mu = 0.15$; the selection script supports additional orders. Single-host repeated timings are descriptive, not corrected multi-seed inferential evidence or forecast-horizon validation. Source rows, mappings, and platform metadata are archived in `paper/benchmarks/selection_coordinate_reference/`.

5.7 Complete normalized-smoothness domain across solvers and criteria

The computational comparison is extended from finite interior penalty grids to the *entire* normalized smoothness domain $S \in [0, 1]$, with 17 candidate values: eleven deciles $0, 0.1, \ldots, 1$ and $0.95, 0.99, 0.999, 0.9999, 0.999999$, and 0.999999999999. All numerical algorithms receive *exactly the same* mapped penalties $\lambda_j = S^{-1}(s_j)$. This preserves the underlying criterion while permitting a direct test of algebraic robustness in the high-smoothness tail.

We examine seven backend implementations: fresh and cached dense linear solves, general banded solves, dense Cholesky, banded Cholesky, symmetric eigendecomposition, and full SVD. For each, the five statistical criteria are classical LOOCV, GCV, AICc, BIC and conditional Gaussian profiled marginal MLE. Difference orders

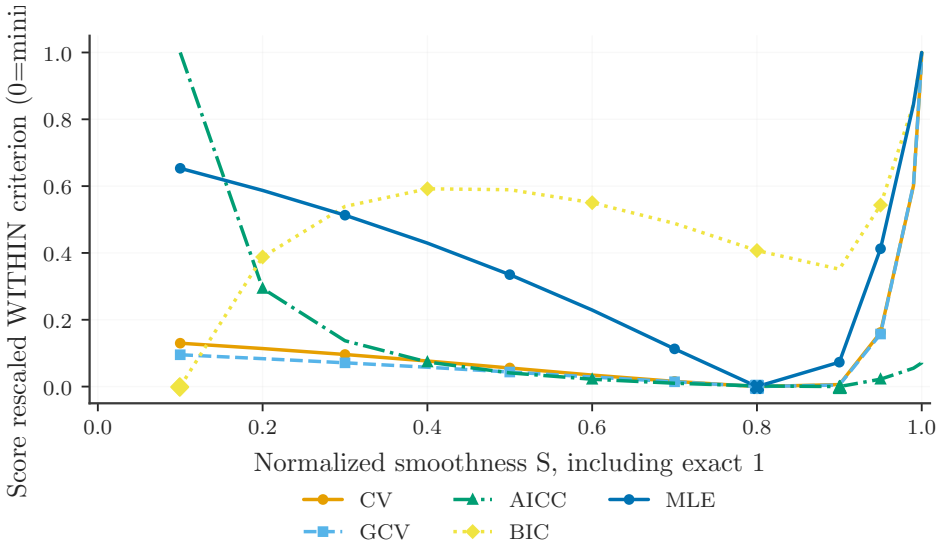


Fig. 6 CV, GCV, AICc, BIC and conditional profiled marginal MLE score profiles on the same mapped smoothness targets for a seeded synthetic series ($n = 60$, $d = 2$, fixed $\mu = 0$). Each profile is shifted and rescaled *within its own criterion* to display shape and minima; ordinate values cannot be compared across criteria. The exact $S = 1$ limit is evaluated.

$d = 0, 1, \ldots, 5$ and fixed difference reference means $\mu = 0$ and $\mu = 0.15$ are included. The public MLE model deliberately excludes $d = 0$; its combinations are reported *not applicable*, not scored or imputed.

Two exact limits cannot be treated as ordinary finite factorization benchmarks. At $S = 0$, $\lambda = 0$ reproduces the observations, making standard LOOCV and GCV undefined for the interpolation-only fit. The marginal likelihood likewise excludes the improper $\lambda = 0$ limit. At $S = 1$, $\lambda = +\infty$: for $d \geq 1$ the fitted trend is the projection onto the polynomial null space of D_d, adjusted for the fixed μ, whereas for $d = 0$ the trend approaches $\mu\mathbf{1}$. All solvers share these *analytical* endpoints; they have no finite Cholesky or LU runtime assigned to them.

For interior penalties, each backend evaluates the complete objective with its own fit and necessary diagnostics: a conditional leverage diagonal for LOOCV, EDF for GCV/AICc/BIC, and a properly normalized determinant/quadratic form or spectral marginal likelihood for MLE. MLE is not charged for unnecessary CV leverage calculations. Candidate scores are tested against a spectral reference outside the timed block. An unstable factorization or an inaccurate score at an extreme finite penalty is explicitly distinguished from a score that the criterion defines as infinite. In particular, a short time from an implementation that fails high-smoothness candidates must *not* be claimed as a valid whole-grid speed victory.

These exploratory timings are tied to the archived host, random seed, sample size and repetition counts in `paper/benchmarks/full_s_reference/environment.json`.

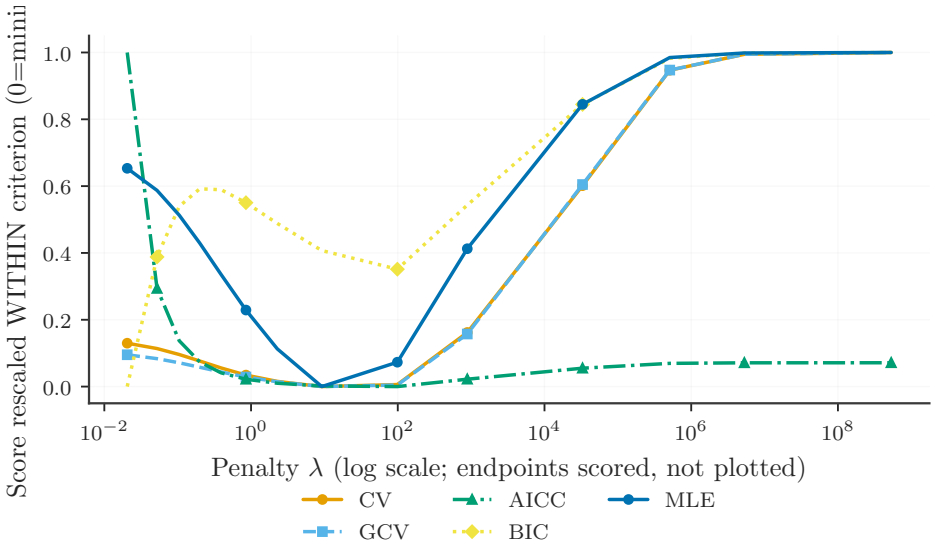


Fig. 7 The same five within-criterion rescaled selection profiles as in Fig. 6, shown against finite positive penalties on a logarithmic axis. Exact $\lambda = 0$ and $\lambda = \infty$ were *scored* but cannot be placed at finite log-axis coordinates; the S-domain figure includes them. This is a change of horizontal coordinate, not of candidate objective.

The exhaustive factorial contains all seven solvers, five criteria, two means and six difference orders, with no invented results for unsupported or unstable combinations. The full candidate-score matrix, numerical failures and method-specific timings are stored in `candidate_scores_and_times.csv`, the complete-grid validity and selection outcomes in `full_grid_summary.csv`, and coordinate mappings in `s_lambda_grid.csv`. The result is a characterization of *conditional in-sample criteria and numerical implementations*, not evidence of improved forecast accuracy or a universally fastest backend.

5.7.1 Single evaluations, repeated grids, and coordinate optimizers

A further experiment refines the previous full-S factorial by separating three computational workloads: (i) one score evaluation at $S = 0.5$, (ii) one complete seventeen-point grid search, and (iii) repeated complete searches on independent series visiting the same candidate penalties. To distinguish factorization reuse from the search coordinate, repeated workloads are evaluated both with cold backend construction on every series and with one shared backend preparation across the series.

For the *discrete* search, we distinguish: (1) *native* S, where each candidate is inverted inside the search, (2) *premapped* $S \to \lambda$, where the same normalized-S candidate set is inverted once and supplied as penalties, and (3) *direct log-λ grid*, with the same finite endpoints, same seventeen candidate count and exact zero/infinite boundaries but different interior candidates. The first two evaluate exactly the same criterion

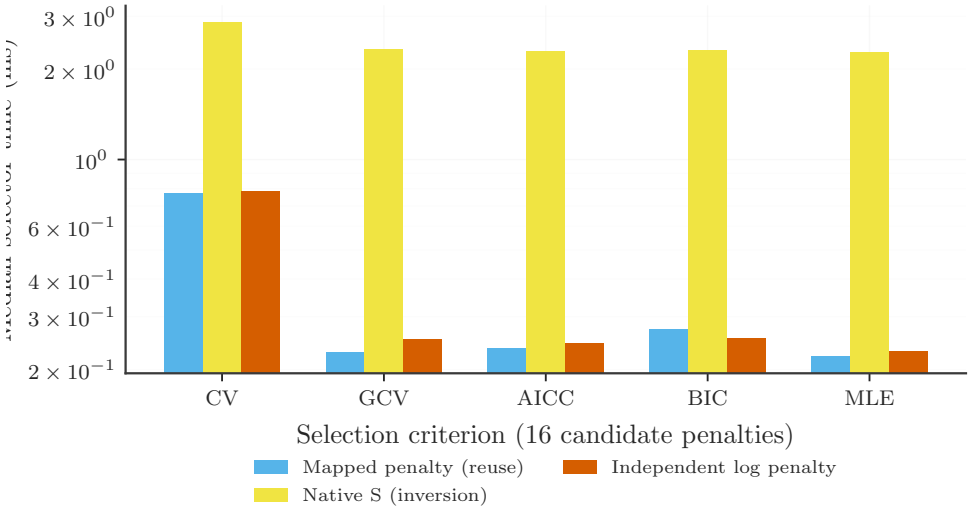


Fig. 8 Measured public spectral `select_lambda` time over 16 candidate points, including MLE and four classical criteria ($n = 60$, $d = 2$, fixed $\mu = 0$). Mapped λ and native S use identical penalties but the latter repeats numerical inversion. The independent log-penalty grid has different interior values. The logarithmic ordinate excludes common spectral setup and shared inversion preprocessing (recorded in separate CSV fields).

scores, up to numerical error: a difference in elapsed time cannot be a new estimator or a statistical advantage. The standalone premapped-grid time includes one initial inversion pass; the backend-only time excludes that common, reusable pass. When multiple criteria and solvers share a single mapped grid its initial inversion should be amortized once, rather than charged separately to each of the thirty-five method–criterion cells.

We additionally compare true *bounded local continuous search* using numerical optimization of $C(S^{-1}(s))$ versus $C(\exp(x))$ in $x = \log \lambda$. The bounds are the common strictly positive finite candidate range and both exact endpoints are scored analytically. These optimizers have variable numbers of evaluations; they are not guaranteed to find a global minimum of a nonconvex criterion. Their minima must not be conflated with a complete grid search, and a local optimizer's different selected smoothness is not by itself a numerical error. CV/GCV/AICc/BIC/MLE retain their original definitions in all five coordinate policies.

The new benchmark preserves the seven original numerical backends, all five criteria, $d = 0, \ldots, 5$ with Gaussian marginal MLE at $d = 0$ explicitly not applicable, fixed $\mu = 0$ and $\mu = 0.15$, and the entire seventeen-point S grid with the exact $S = 1$ limit. Numerical agreement with the spectral score is checked *outside* the timed calculations; direct methods that lose accuracy in the extreme near-one tail do not qualify for a valid whole-grid speed ranking. Backend construction, evaluation, inversion, and standalone total are stored separately. One isolated call is not evidence for how repeated searches amortize factorizations.

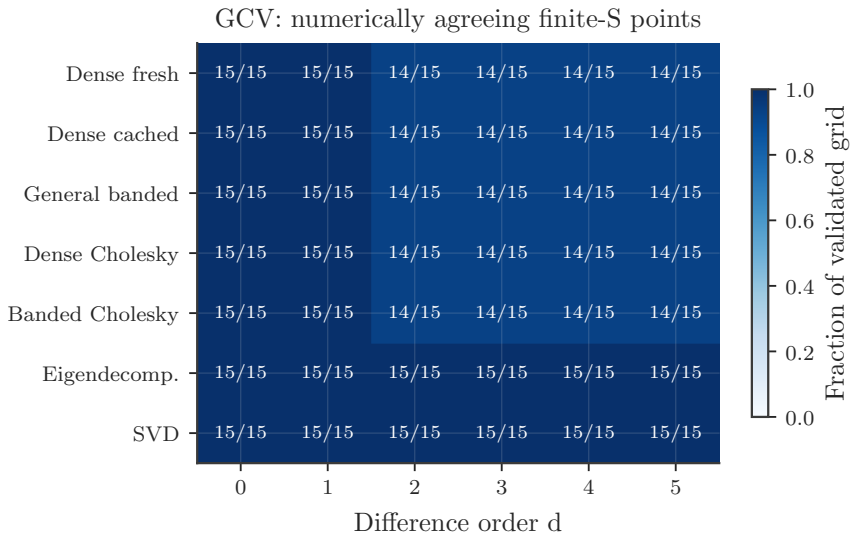


Fig. 9 Numerical agreement over all fifteen *finite interior* normalized-S candidates in an actual GCV selection sweep, for $n = 24$, fixed $\mu = 0$, and $d = 0, \ldots, 5$. Every heatmap cell shows the number of backend scores agreeing with the reference out of fifteen; the exact endpoints are handled analytically. The plot separates arithmetic stability from elapsed time.

The companion website shows further plots of cold/shared solver preprocessing and selected S for each coordinate strategy. Raw measurements, valid/invalid result flags, evaluation counts, and machine metadata are stored in `paper/benchmarks/full_s_search_reference/`. The figures are generated with the same AwesomeFigures Springer-width vector pipeline. As with the previous factorial, these in-sample measurements do not establish out-of-sample forecasting superiority or a universally fastest optimizer.

In the recorded hosted-Linux quick study, using the spectral implementation with $n = 24$, $d = 2$, fixed $\mu = 0$ and two timing repetitions, an isolated GCV search over the full seventeen-point normalized-S grid required 2.113 ms when inverting targets inside the search and 2.019 ms when the same grid was premapped once *including* that mapping cost. The corresponding single GCV score at $S = 0.5$ took 0.374 ms with native S, compared with 0.144 ms at a supplied known penalty. For five criteria revisiting exactly the same grid, native-S standalone times summed to 10.872 ms; shared preparation was 1.627 ms and the five combined backend-only selection times 2.278 ms, a total of approximately 3.905 ms. This 2.78-fold descriptive difference illustrates why amortized inversion matters even when isolated full-grid times are similar. It is not an inference about other series sizes or machines.

For GCV, the bounded local optimizers selected nearly the same smoothness ($S \simeq 0.695064$), requiring 1.826 ms in S and 0.554 ms in $\log \lambda$ on this example. However, the BIC local optimizers converged to distinct candidate smoothness values ($S \simeq 0.1$

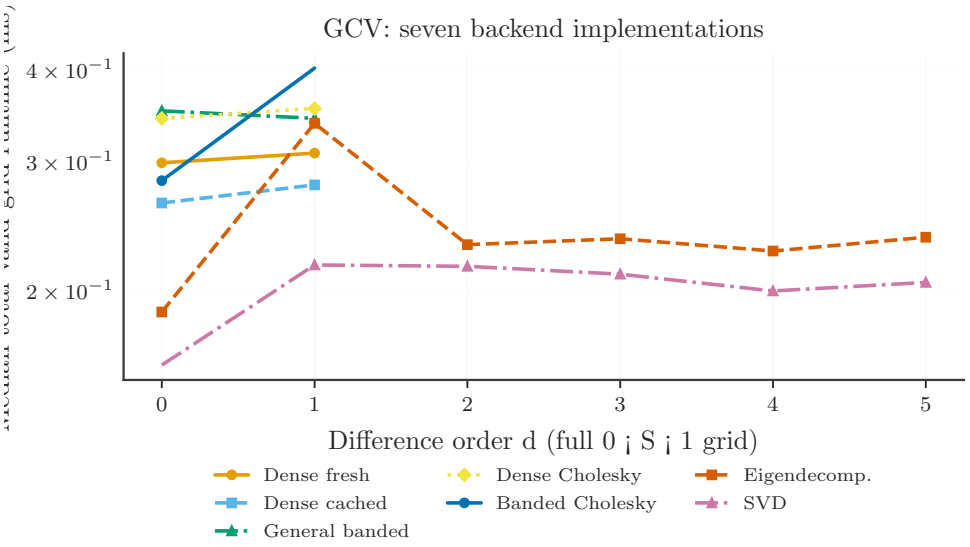


Fig. 10 Measured total method-specific runtime for an entire GCV grid, $n = 24$, $\mu = 0$, and varying difference order. The cost includes backend preparation and scoring all fifteen finite interior penalties, with the same λ values for each solver. A backend is *omitted* at a given order if it fails the numerical agreement or finite-score test for any interior point; therefore this figure must be read together with Fig. 9. The vertical axis is logarithmic.

and $S \simeq 0.699$), reinforcing that bounded local minimization is not a global criterion optimization guarantee. Every figure and timing value refers to the archived runner and a specified workload, not to an abstract superiority of either coordinate.

5.8 Author-workstation full-factorial search results

The numerical investigations were extended using the author's complete Windows-machine `full` run. The original archived experiment contains 80 640 search records: $n \in \{24, 48\}$, difference orders $d = 0, \ldots, 5$, the two fixed means $\mu = 0$ and $\mu = 0.15$, five selection criteria (CV, GCV, AICc, BIC, conditional marginal MLE), seven numerical solver implementations, and $R \in \{1, 3, 8\}$ independent observed series, with cold and shared backend preparation. The eight computational policies distinguish one evaluation, three complete 17-point finite-grid searches, and two bounded *local* continuous searches. Two seeds were executed and every configuration was timed with seven repetitions and one BLAS thread. This is a separate physical machine and experimental design from the earlier hosted-Linux reference figures.

The normalized-smoothness grid includes the exact $S = 0$ and $S = 1$ endpoints together with an increasingly fine near-one tail through $S = 0.999999999999$. The exact $S = 1$ endpoint corresponds to $\lambda = \infty$ and is scored analytically; it is not a request to factor an infinite-penalty matrix. Numerical instability at extreme finite

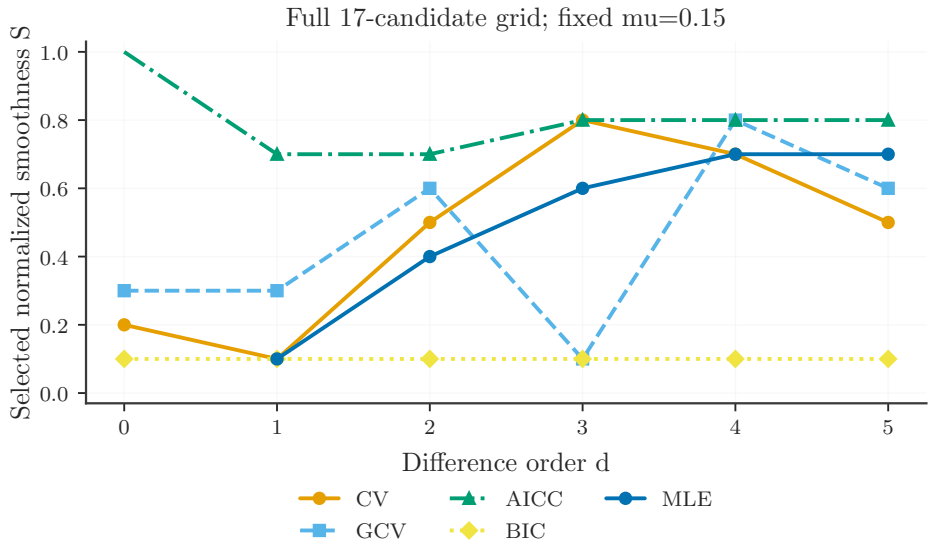


Fig. 11 Selected normalized smoothness across all five criterion definitions and difference orders at fixed nonzero $\mu = 0.15$. Selections are made from the *entire* seventeen-point grid, including the analytical $S = 1$ endpoint. MLE at $d = 0$ is not applicable under the implemented likelihood assumptions. Selected criterion values are not themselves comparable across different statistical objectives.

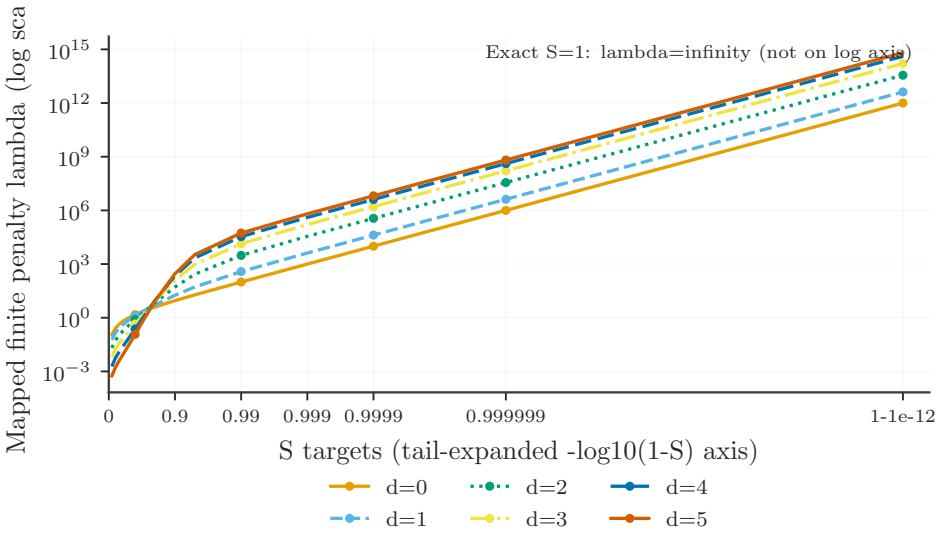


Fig. 12 The distinct map $S \mapsto \lambda$ for difference orders $d = 0, \ldots, 5$ over the shared near-one candidate grid. The vertical axis shows only finite positive penalties, on a logarithmic scale. The *exact* $S = 1$ endpoint maps to $\lambda = +\infty$ and therefore cannot appear on that axis.

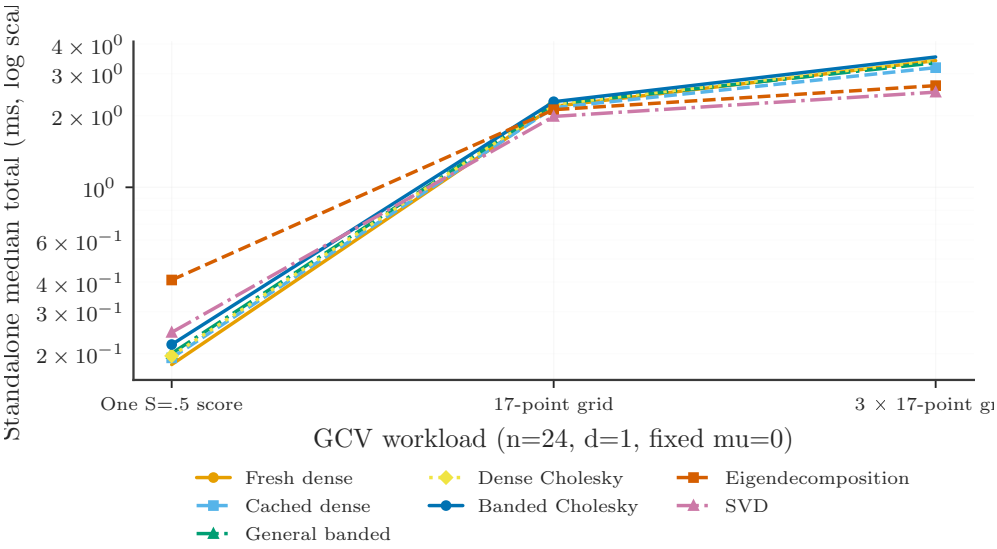


Fig. 13 Measured work for one $S = 0.5$ GCV score, one seventeen-point full-S grid search, and three independent full-grid searches at $n = 24$, $d = 1$ and fixed $\mu = 0$. All curves use identical mapped penalties, with shared backend preprocessing, and the standalone mapped-grid inversion charged once. Only numerically validated method/workload results are shown. Ordinate logarithmic.

penalties and statistical inapplicability are recorded separately; MLE for $d = 0$ is outside the adopted likelihood model.

Both the *native* S grid and the *premapped* $S \to \lambda$ grid sample the identical 17 candidate penalties, so they must agree in criterion scores and selected smoothness. The independently spaced log-penalty grid uses equal candidate count and outer finite endpoints but *different interior candidates*. Only the two matched grids can support a pure computational-speed comparison at a fixed discrete statistical estimator. For pretranslation we charge the one-time inverse-grid construction exactly once to a standalone search; repeated cross-criterion uses can amortize it.

When reusing exactly one pretranslated candidate grid across all five criteria, the measured native-S total was 11.234 ms and the shared $S \to \lambda$ mapping plus five backend-only searches took 3.980 ms (descriptive ratio 2.823). This accounting charges grid inversion exactly once, not once per statistical criterion.

The 80 640 original author-host measurements and machine metadata are preserved under `paper/benchmarks/results/full_s_search_modes/`. The Windows study covers $n = 24, 48$, $d = 0, \ldots, 5$, $\mu = 0, 0.15$, $R = 1, 3, 8$, seven backends, five criteria, and cold versus shared backend preparation. MLE at $d = 0$ remains not applicable. At $S = 1$ the $\lambda = \infty$ limit is evaluated analytically.

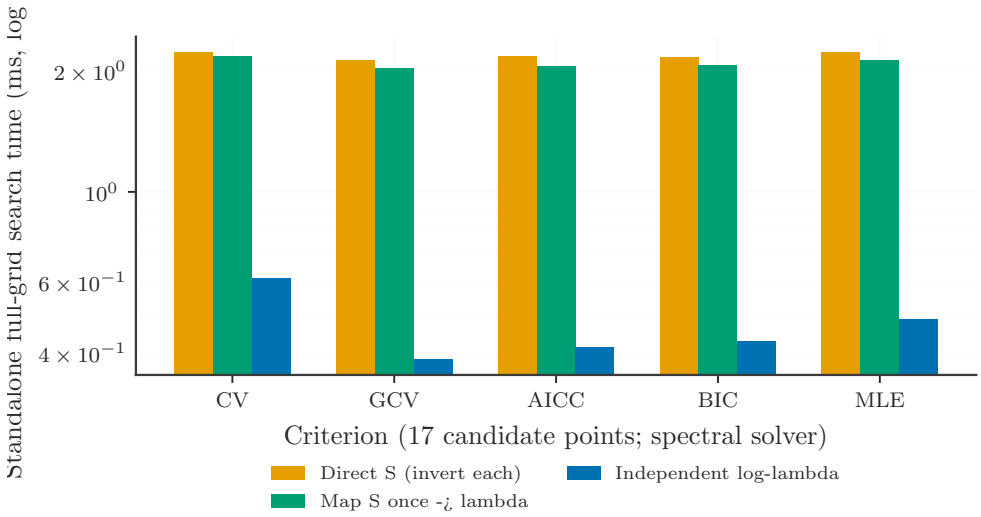


Fig. 14 Standalone measured complete-grid selection time for CV, GCV, AICc, BIC and Gaussian marginal MLE on a spectral solver, $d = 2$, $\mu = 0$, $n = 24$. Native S and premapped $S \to \lambda$ sample the *same* seventeen candidates. An independent log-λ grid has the same endpoints and budget, but different interior candidates and potentially different selected scores. The one-time premapping is included in its standalone timing. Ordinate logarithmic.

All timings are empirical medians under the recorded Windows machine and software configuration; the source CSV, grid-construction records, and environment metadata are retained unmodified. Automatically derived two-seed aggregates, selected-parameter summaries, numerical parity checks, the data-driven table above and all AwesomeFigures PDF/SVG artwork are committed separately under `paper/benchmarks/windows_full_s_analysis/`. They neither supersede the hosted-Linux reference measurements nor demonstrate improved out-of-sample forecast accuracy, a new smoothing criterion, or a universally optimal numerical backend.

5.9 A targeted three-backend engineering check

The normal-equations matrix $A_\lambda = I_n + \lambda D_d^\top D_d$ is symmetric positive definite at finite $\lambda \geq 0$ and has half-bandwidth d. Consequently, direct banded Cholesky requires $O(nd^2)$ arithmetic and $O(nd)$ storage for one factorization. A sparse Cholesky package can exploit the same one-dimensional sparsity, but its generic symbolic machinery need not outperform a dedicated banded implementation. In contrast, full spectral decomposition pays $O(n^3)$ preprocessing and $O(n^2)$ storage, while reusable projected scalar criteria such as GCV, AICc and residual BIC can be evaluated in $O(n)$ per interior penalty candidate. Full fitted trends and ordinary leave-one-out leverage calculations have additional costs. These distinctions concern *complete criterion pipelines*, not merely a single linear solve.

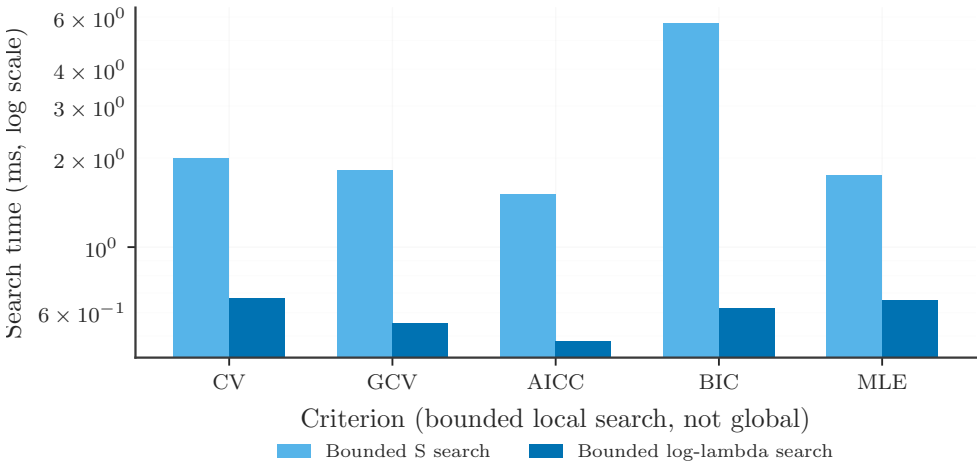


Fig. 15 Measured bounded local objective minimization directly in normalized smoothness and in $\log \lambda$, with exact $S = 0$ and $S = 1$ endpoints considered separately. These use variable function-evaluation counts and are *not* statistically equivalent to searching exactly the seventeen-point grid or certified global optimization. Same fixed mean and series as Fig. 14.

To check functional reproducibility, we reconciled two archived Windows 11 numerical batches for the same $(n, d) = (64, 1)$, conditional plug-in reference mean $\mu = \overline{D_1 y}$, high-amplitude synthetic series, residual BIC criterion, and an $M = 100\,000$ target-smoothness grid. The three complete pipelines were dense naive inversion, reusable spectral SVD and genuine sparse CHOLMOD Cholesky followed by separately charged

Table 7 Author's Windows full-run median complete 17-point grid selection times (milliseconds) for two independent recorded seeds, $n = 48$, $d = 2$, fixed $\mu = 0$, $R = 1$, spectral SVD, one BLAS thread and seven timing repetitions. Premapped $S \to \lambda$ includes one complete grid conversion. Independent log-λ has different interior nodes. Cells are shown only where both seeds are numerically valid.

Criterion	Native S	Premapped λ	Log-λ
CV	2.004	1.879	0.545
GCV	1.851	1.800	0.426
AICc	1.887	1.810	0.458
BIC	2.016	1.810	0.449
MLE	3.477	2.035	0.515

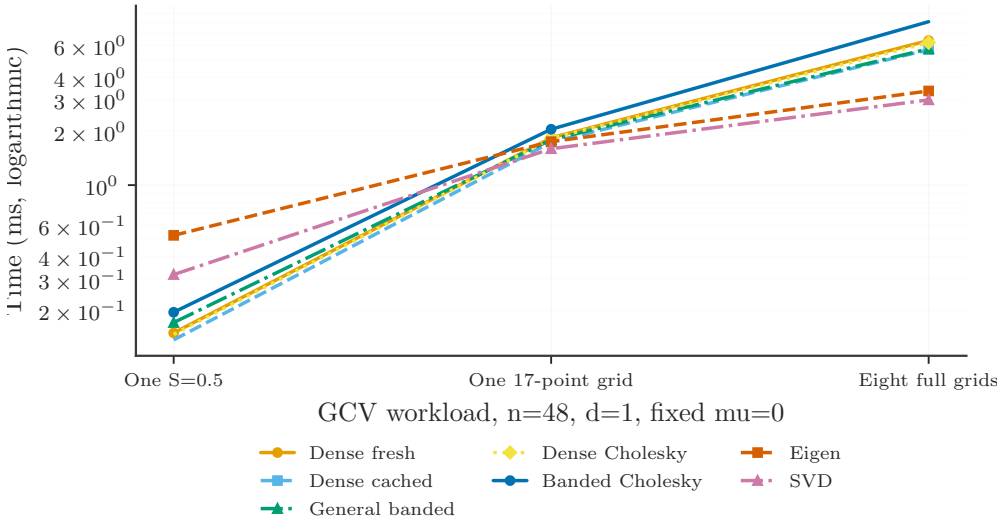


Fig. 16 Full Windows-run comparison of one GCV evaluation at $S = 0.5$, one complete seventeen-point search and eight independent complete searches ($n = 48$, $d = 1$, $\mu = 0$). The vertical axis is logarithmic and each backend is included only for workloads that were numerically valid in both recorded seeds. Premapped standalone times include the common grid inversion once.

Table 8 Observed median cold total time, in seconds, for the $n = 64$, $d = 1$, $M = 100\,000$ conditional plug-in BIC check, using a pretranslated smoothness grid where relevant. Each cell is the median of two archived repeats. The SVD and naive-inverse runs used a pool of eight concurrent method–case processes; the later CHOLMOD backfill used one worker. The times are therefore *descriptive*, not controlled comparative speedups.

Workload	Spectral SVD	Naive inverse	CHOLMOD hybrid
One candidate, one series	0.00195	0.00113	0.00270
10^5 candidates, one series	10.226	15.167	31.533
One candidate, two series	0.00189	0.00102	0.00274
10^5 candidates, two series	11.686	20.640	37.144

banded selected-inverse diagnostics. The normal-equations solver, smoother, and statistical criterion were not changed. Each implementation produced 16 valid rows across two repeats, two coordinates, and four one/multiple-candidate by one/multiple-series workloads. In total, all 48 recorded observations had successful numerical status. Both repeated outcomes selected $S = 10^{-5}$ for the grid workloads, consistent with the known near-interpolation behavior of the adopted residual BIC expression. This boundary choice should not be interpreted as evidence of a statistically well-identified interior optimum.

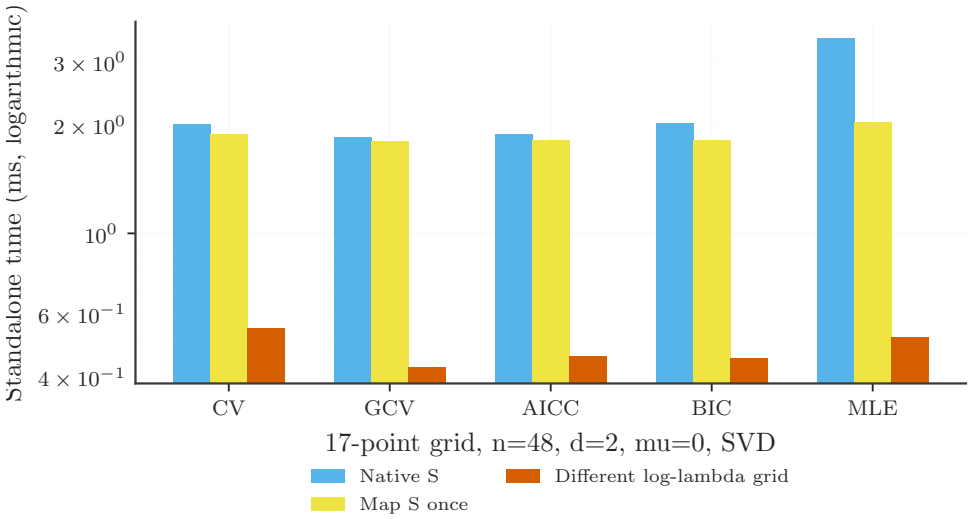


Fig. 17 Observed Windows elapsed time for five existing selection criteria under native S, a once-premapped S grid evaluated in λ, and an independently sampled log-λ grid ($n = 48$, $d = 2$, $R = 1$, $\mu = 0$, spectral SVD). Premapping is charged once to the standalone time. The third policy samples different interior penalty candidates, so runtime alone does not establish statistical superiority. The vertical axis is logarithmic.

The maximum relative difference in selected BIC score from the SVD reference across the same 16 workload–coordinate–repeat keys was 2.321×10^{-12} for naive inversion and 4.463×10^{-13} for the sparse-Cholesky hybrid, with identical selected smoothness at the recorded resolution. These *float64 selected-score parity* checks are not independent high-precision solution-error certificates. CHOLMOD's first archived attempt had failed at dependency initialization; the separately archived successful backfill uses the installed scikit-sparse implementation. The short hardware timings are not pooled with the distinct historical 80 640-record factorial experiment.

The complete 48-row CSV, original source revisions, worker counts, seed, workload definitions and an executable validation script are preserved under `paper/benchmarks/experiment010/publication_evidence/`. For a controlled wall-time comparison, the three methods must be rerun under identical CPU contention, numerical tolerances and caching conditions. Merely setting `Workers=8` on a single method–case task does not parallelize its factorization: the worker pool schedules independent tasks, not partitions of one Cholesky solve. We do not extrapolate this $n = 64$ case to a universal solver cutoff, or require a much larger brute-force grid to establish the theory of the estimator.

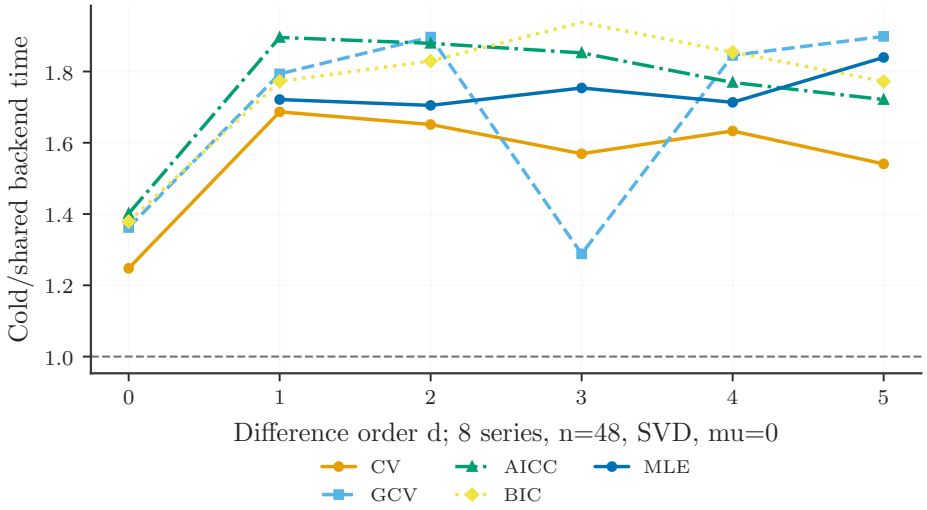


Fig. 18 Cold-to-shared backend-only runtime ratio for repeated $R = 8$ full-grid searches across orders $d = 0, \ldots, 5$ ($n = 48$, fixed $\mu = 0$, SVD). The ratio is shown only when both experimental seeds satisfy each criterion's numerical validity requirements. Shared spectral preprocessing can be amortized across series, but does not make the grid itself or matrix factorization cost free.

6 A controlled exploration of larger difference orders

The software accepts any integer $0 \leq d < n$, but this algebraic admissibility must be distinguished from floating-point stability. We therefore conduct a deterministic diagnostic with $n = 50$, seed 24021, difference orders $d = 1, \ldots, 10$, $\mu = 0$, and common normalized smoothness $S = 0.86$. At each order, we form the full SVD of D_d, compute the condition ratio

$$\kappa_+(Q_d) = \frac{\max_{1 \leq j \leq n-d} \delta_j}{\min_{1 \leq j \leq n-d} \delta_j}, \tag{27}$$

and compare the spectral fitted trend with an independent direct solution of $(I + \lambda Q_d)\tau = y$. The ratio is calculated from the *strictly positive* penalty eigenvalues; the full Q_d has zero eigenvalues and is singular for every $d \geq 1$.

The ratio rises from about 10^3 at $d = 1$ to 2.44×10^{15} at $d = 10$. At $d = 10$, the maximum componentwise difference between the SVD-based fit and the direct normal-equation solution is approximately 3.36×10^{-7} for the declared sample and smoothing target, versus approximately 1.14×10^{-15} at $d = 1$. The direct normal-equation result is not presumed an exact ground truth at large condition numbers: both approaches are affected by finite precision. The experiment illustrates why support for many difference orders needs accompanying conditioning diagnostics, rather than establishing new high-order theory, unconditional stability, or performance superiority. Machine-specific setup/fitting times are separately exported by the experiment script for later audit; they are not promoted to general speed benchmarks.

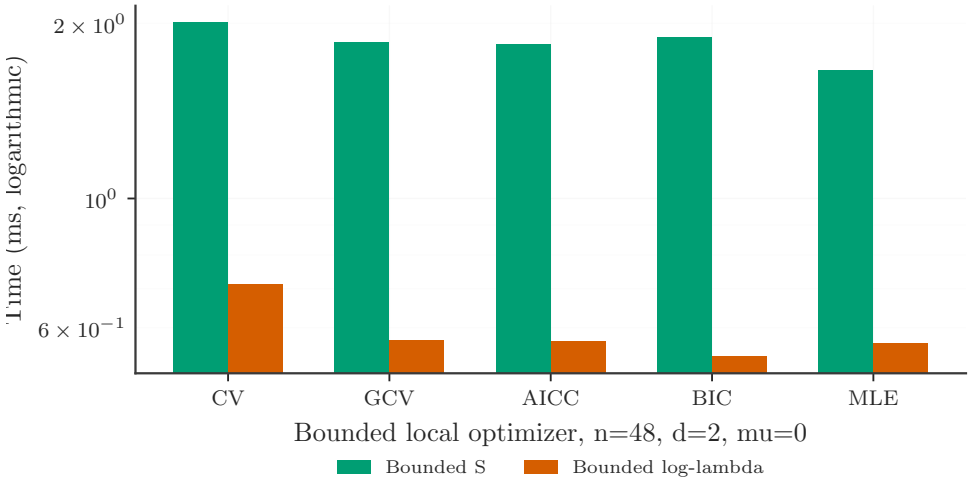


Fig. 19 Measured bounded *local* scalar optimization in the smoothness coordinate S and in $\log \lambda$ ($n = 48$, $d = 2$, $\mu = 0$, SVD), including analytical endpoint checks. Function-evaluation counts and candidate positions can differ; a bounded optimizer does not certify a global criterion minimum. The ordinate is logarithmic.

7 Observed-data illustration: atmospheric carbon dioxide

We update the Mauna Loa atmospheric CO_2 illustration through the most recently released complete monthly observation available on October 25, 2026. The NOAA Global Monitoring Laboratory (GML) monthly means cover January 1990–September 2026, $n = 441$ equally spaced months; the September 2026 mean is 426.07 parts per million (ppm) NOAA Global Monitoring Laboratory (2026). The October 2026 monthly mean was not available at the data cutoff. We analyze the *published monthly means*, without estimating missing observations or removing the substantial annual seasonal fluctuations.

These data differ in provenance from the archived 1990–1999 example, which aggregated the historical weekly series distributed by `statsmodels.datasets.co2`. The original 120-month data, figure, and reproduction script remain in the repository, but the updated plot uses NOAA's monthly averages directly. Values for 1990–August 2026 were transcribed from a public mirror of NOAA's monthly series and the September 2026 value was checked against NOAA's October 9 release. The exact 441-value slice, source locations, and cutoff date are recorded in `figures/noaa_mlo_monthly_1990_2026_09.csv`. NOAA values are subject to subsequent revision.

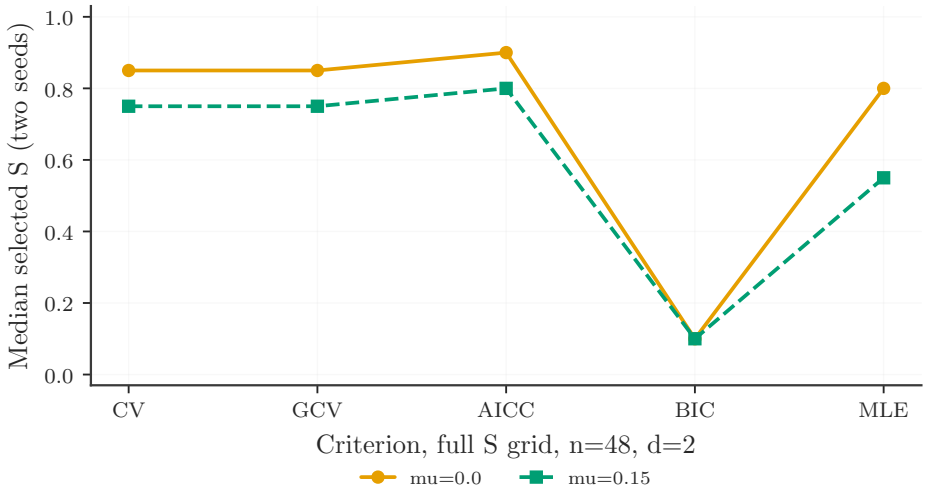


Fig. 20 Median selected normalized smoothness for each statistical selection criterion under the *same mapped* seventeen-point candidate grid, contrasting fixed $\mu = 0$ and $\mu = 0.15$ ($n = 48$, $d = 2$, spectral SVD, two experimental seeds). Only validated solutions appear; the criterion values themselves have different scales and must not be compared across criteria.

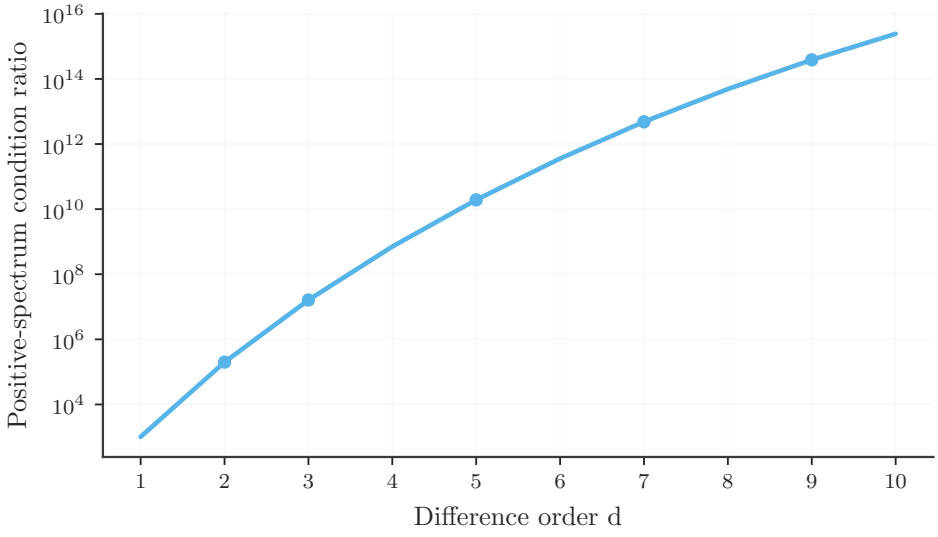


Fig. 21 Condition ratio of the positive spectrum of $D_d^\top D_d$ for $n = 50$, $d = 1, \ldots, 10$. Values are recomputed from singular values of D_d; they do not represent hardware-specific runtime.

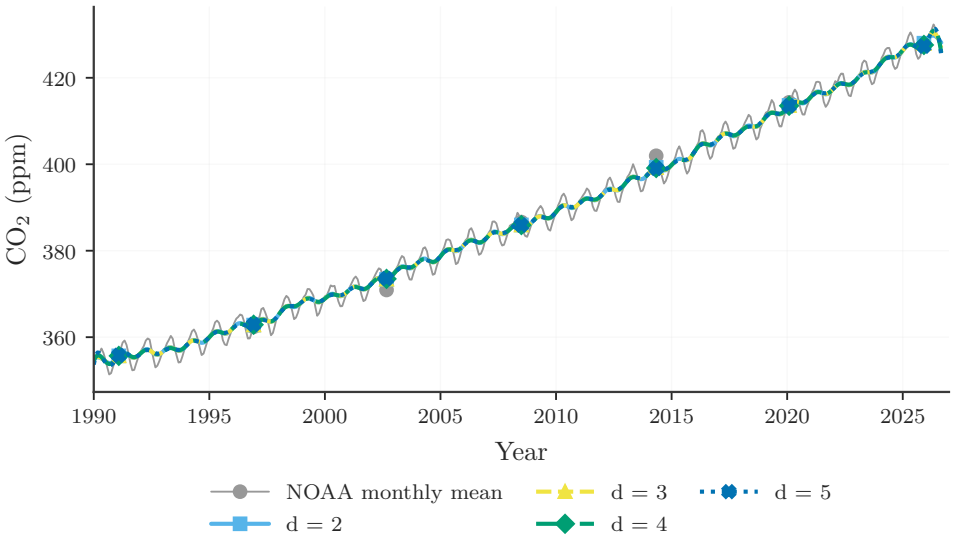


Fig. 22 NOAA GML Mauna Loa monthly-mean atmospheric CO_2 (1990–September 2026; $n = 441$) and two-sided PLS estimates of orders $d = 2, 3, 4, 5$, each with fixed $\mu = 0$ and normalized smoothness $S = 0.86$. The original shorter 1990–1999 illustration remains reproducible in the source repository. These are historical descriptive fits, not forecasts.

As in the archived illustration, we fix the reference difference mean at $\mu = 0$, prescribe the same normalized conditional trace smoothness $S = 0.86$, and compare orders $d \in \{2, 3, 4, 5\}$. For each order we invert Eq. (17), then fit the trend with the recovered penalty. The new penalties λ are, respectively, approximately 41.77, 175.72, 787.53, and 3564.46. The associated conditional effective degrees of freedom, $\nu = n - (n-d)S$, are 63.46, 64.32, 65.18, and 66.04. Unlike a common numeric penalty, a common S controls the *fraction of penalizable trace degrees of freedom* across orders; it does not guarantee identical geometric trends.

The observed rising CO_2 baseline and seasonal oscillations illustrate why a smoothed historical trend is not automatically an estimate of measurement noise or a seasonally adjusted signal. Different orders preserve different polynomial null spaces and can behave differently near the endpoints. Using the full historical series gives a *two-sided* trend estimate: observations later than a given month enter that month's fitted value. This comparison does not estimate μ from observations or optimize a Gaussian likelihood; no held-out future period or forecast accuracy is evaluated. The displayed two-sided fitted curves are not forecasts.

A substantive atmospheric analysis would explicitly model annual seasonality, site changes, measurement uncertainty, and later data revisions. These tasks, like statistical inference for a jointly estimated (λ, μ), remain outside the present computational comparison.

8 Discussion and conclusions

The normalized trace index $s = S(\lambda)$ provides a bounded, interpretable coordinate for the same penalized least-squares estimators and classical selection criteria used in penalty space. Monotonicity and the inverse mapping preserve ideal criterion minimizers, while the chain rule transforms the analytical CV derivative. The public implementation offers both coordinates. Regression checks verify agreement on mapped candidate grids, including boundaries and nonzero fixed difference means; they do not establish uniform numerical stability or universal convergence for arbitrary inputs.

The reparameterization has distinct computational costs. Each interior s typically requires a numerical inversion to recover λ, and the derivative conversion can be sensitive near $s = 0$ and $s = 1$. A bounded search interval therefore provides interpretability rather than an automatic speed improvement. Measured end-to-end finite-grid GCV searches at matched candidate budgets compare both coordinates under controlled penalty endpoints, but finite grid discretization and the extra inversion work prevent a universal recommendation for either sampling policy.

Our complementary solver comparisons distinguish one-time preprocessing, repeated scalar trace evaluations, and complete fitted-trend reconstruction. A reusable spectral decomposition provides cheap scalar evaluations after preprocessing, but a dense full SVD incurs $O(n^3)$ setup time and $O(n^2)$ storage, and banded methods remain competitive, particularly for isolated fits. The new six-backend GCV measurements under fixed zero and nonzero reference means support spectral computation for criterion selection in the tested workloads, while banded Cholesky is an efficient choice for many full- trend reconstructions. Reusing Cholesky factors is beneficial only when the same penalty candidates recur across multiple series. Observed timing differences apply to the stated workloads and hardware. The additional three-backend verification at $n = 64$ exhibits close selected-BIC agreement, but the CHOLMOD backfill was performed with a different number of concurrent workers; its measured seconds are therefore descriptive rather than an unconfounded head-to-head speed ranking. A banded Cholesky factorization exploits the d-banded positive-definite system at $O(nd^2)$ cost; the observed CHOLMOD hybrid also pays for exact selected-inverse diagnostics. Higher differencing orders can amplify numerical conditioning problems; a common s across orders standardizes a conditional trace index, *not* trend geometry.

The nonlinear synthetic example illustrates different selections from established CV, GCV, AICc, and BIC criteria, without a general method ranking. The atmospheric carbon-dioxide case illustrates historical smoothness control, not future forecast accuracy. Our contribution is an auditable numerical treatment of an interpretable coordinate for an existing PLS family, supported by explicit solver trade-offs; it is not a new statistical penalty, a new likelihood theory, or evidence of forecast superiority. Joint estimation of (λ, μ) and inferential comparisons remain outside the scope of the present study.

Declarations

Funding. To be confirmed by the authors before submission.

Competing interests. To be confirmed by the authors before submission.
Ethics approval and consent. Not applicable to the presented numerical algorithms and historical aggregate time-series examples; authors to verify.
Data availability. The simulated data can be reproduced from the supplied scripts and seeds. The observational monthly carbon-dioxide source is a frozen NOAA Mauna Loa snapshot through September 2026, with a documented derived-data provenance in the repository. The numerical benchmark rows, their original GitHub results branches, frozen CSV, and validation script are preserved under `paper/benchmarks/experiment010/publication_evidence/`.
Code availability. The implementation and paper-specific simulation and benchmark scripts are maintained at https://github.com/heri-espino/trendestimation. A permanently archived release identifier should be added before submission.
Author contributions. To be confirmed and approved by both authors before submission.

References

Biessy, G.: Whittaker–henderson smoothing revisited: A modern statistical framework for practical use. ASTIN Bulletin: The Journal of the IAA **56**(1), 1–31 (2026) https://doi.org/10.1017/asb.2025.10061

Cortés-Toto, D., Guerrero, V.M., Reyes, H.J.: Trend smoothness achieved by penalized least squares with the smoothing parameter chosen by optimality criteria. Communications in Statistics—Simulation and Computation **46**(2), 1492–1507 (2017) https://doi.org/10.1080/03610918.2015.1005236

Guerrero, V.M.: Time series smoothing by penalized least squares. Statistics & Probability Letters **77**(12), 1225–1234 (2007) https://doi.org/10.1016/j.spl.2007.03.006

Guerrero, V.M.: Estimating trends with percentage of smoothness chosen by the user. International Statistical Review **76**(2), 187–202 (2008) https://doi.org/10.1111/j.1751-5823.2008.00047.x

NOAA Global Monitoring Laboratory: Trends in Atmospheric Carbon Dioxide: Monthly Average at Mauna Loa Observatory. Global Monitoring Laboratory, National Oceanic and Atmospheric Administration. Data release October 9, 2026; September 2026 is the latest complete monthly mean at the October 25, 2026 cutoff. Monthly-average values may be revised (2026). https://gml.noaa.gov/ccgg/trends/

Weinert, H.L.: Efficient computation for whittaker–henderson smoothing. Computational Statistics & Data Analysis **52**(2), 959–974 (2007) https://doi.org/10.1016/j.csda.2006.11.038

Weinert, H.L.: Fast Compact Algorithms and Software for Spline Smoothing. SpringerBriefs in Computer Science. Springer, New York (2012). https://doi.org/10.1007/

978-1-4614-5496-0 . Electronic edition published October 2012; Springer copyright 2013

Whittaker, E.T.: On a new method of graduation. Proceedings of the Edinburgh Mathematical Society **41**, 63–75 (1923) https://doi.org/10.1017/S0013091500077853